appendix **C**

Programming UFOs

What is a Process UFO?

A Media Illusion Process User Function Object (UFO) is code that performs a specific image-processing effect. It contains a selection from a set of pre-defined user functions such that it can be loaded by Media Illusion and accessed as a normal Media Illusion process. A term commonly used for this type of package is plug-in.

Who can Write UFOs?

Anybody with experience of programming in C or C++ and has an understanding of the effect to be achieved can write UFOs.

In-house or Third-party UFOs

UFOs can be created for one-off effects or commonly required in-house effects that are not already supplied by Media Illusion. UFOs may also be supplied by third-party producers who specialize in the field of generating marketable special effects.

Implementation

A UFO is written as a set of pre-defined user functions which define the setup and rendering of an effect. These fall into three main areas:

- Process Definition and Initialization
  - Process identification
  - Input, output and parameter definition
  - Process and parameter initialization
- Process Rendering
  - Pre- and post-rendering
  - Defined output and required input region calculation
  - Rendering
- External User Interface
  - Editor screen drawing
  - Editor screen interaction
  - Editor x-motif graphical user interface (GUI)

These user functions communicate with the main body of code via an interface library of support functions. The UFO code is compiled and linked into a Dynamic Shared Object (DSO) which is loaded into the main executable at run-time.

The symbols for the pre-defined functions are searched for in the UFO and their addresses are tabulated for future calling. The final appearance when the UFO is run is as if the functions were directly linked into the product.

The interface library is written with C binding for which the following header files are supplied:

- *ufoProcess.h*

  Process interface and general interface
- *ufoRaster.h*

  Raster access interface, pixel structures, and constants
- *ufoTypes.h*

  General interface constant definitions
- *ufoEditor.h*

  Editor interface

The complete set of user functions is declared in the *ufoFunctions.h* header file. This should be included in the UFO source code files to enforce the correct function protocol.

The user functions are:

Function	Description
`ufoProcessDefine`	process definition and initialization
`ufoProcessCopyUserData`	user data copy
`ufoProcessDeleteUserData`	user data delete
`ufoProcessPreSequenceRender`	pre-render sequence
`ufoProcessPostSequenceRender`	post-render sequence
`ufoProcessCanAvoidRendering`	pre-frame render function to determine if rendering can be avoided
`ufoProcessPreRender`	pre-render
`ufoProcessPostRender`	post-render
`ufoProcessPreRenderPass`	pre-render pass
`ufoProcessPostRenderPass`	post-render pass
`ufoProcessRenderPixel`	pixel render
`ufoProcessRenderLine`	line render
`ufoProcessRenderRectangle`	rectangle render
`ufoProcessCalcDefinedRectangle`	calculate rectangle which can be rendered to output(s)
`ufoProcessCalcNeedRectangles`	calculate rectangles which are required from inputs
`ufoProcessCalcDefinedPixelType`	calculate the “best” pixel type given the supplied input pixel types
`ufoProcessSpecifyConvertPixelTypes`	specify input and output pixel type conversions
`ufoProcessWriteAsciiData`	write user ASCII data string to process
`ufoProcessReadAsciiData`	read user ASCII data string from process
`ufoProcessWriteBinaryData`	write user binary data string to process
`ufoProcessReadBinaryData`	read user binary data string from process
`ufoProcessParamsEdited`	callback routine on parameter edits; can also set other parameters to be changed too

Function	Description
`ufoEditorCreate`	GUI creation and initialization
`ufoEditorDelete`	GUI deletion
`ufoEditorShowGui`	display GUI, such as manage pop-ups
`ufoEditorHideGui`	hide GUI, such as manage pop-ups
`ufoEditorUpdateGui`	update GUI (to show parameter values)
`ufoEditorRGBDraw`	draw into RGB plane of screen
`ufoEditorOverlayDraw`	draw into overlay plane of screen
`ufoEditorMouseDown`	process mouse button down event onscreen
`ufoEditorMouseUp`	process mouse button up event onscreen
`ufoEditorMouseDrag`	process mouse drag onscreen
`ufoEditorMousePosition`	process mouse move (no buttons down) event onscreen
`ufoEditorKeyEvent`	process key event onscreen
`ufoEditorSetParamGroup`	set/change parameter group to be drawn or displayed in GUI

The minimum requirement is that a process UFO defines *ufoProcessDefine*, and one of the following: *ufoProcessRenderPixel*, *ufoProcessRenderLine* and *ufoProcessRender-Rectangle*. If this requirement is not met, then the UFO process cannot be used and will not appear in Media Illusion.

The minimum requirement for screen interaction is that a process UFO defines *ufoEditorCreate*, and one of the following: *ufoEditorRGBDraw*, *ufoEditorOverlayDraw, ufoEditorMouseDown, ufoEditorMouseUp, ufoEditorMouseDrag, ufoEditorMousePosition*, and *ufoEditorKeyEvent*.

The minimum requirement for a user-defined X Motif GUI is that a process UFO defines both *ufoEditorCreate* and *ufoEditorUpdateGui*.

Definition

The definition and initialization of UFO process must be supplied in the *ufoProcessDefine* user function. This includes the definition of the process ID, the number and types of inputs, outputs and parameters, parameter default initialization, and other general initialization.

ufoProcessCreate returns the initial definition instance of a UFO process and subsequent UFO process instances are essentially copied from it or copied from copies of it. This instance is supplied to most functions called within the body of the definition user function. Each copy of a particular UFO process has an instance object associated with it, and therefore in all other UFO user functions and most UFO library functions, a UFO process instance is passed as the first argument.

Options

The number, type and subgrouping of editable parameters must be defined in the *ufoProcessDefine* user function. The available set of parameter types are:

Parameter	Description
`ufoBooleanParam`	true/false, on/off
`ufoEnumParam`	list selection
`ufoIntegerParam`	integer
`ufoFloatParam`	floating point
`ufoNormalisedParam`	floating point normalized (0..1)
`ufoAngleParam`	floating point angle
`ufoColourRGBParam`	RGB colour
`ufoColourCMYParam`	CMY colour
`ufoColourHSVParam`	HSV colour
`ufoColourHLSParam`	HLS colour
`ufoRectangleParam`	floating point rectangle
`ufoPos3dParam`	floating point 3D position
`ufoScale3dParam`	floating point 3D scale
`ufoPos2dParam`	floating point 2D position
`ufoScale2dParam`	floating point 2dD scale
`ufoStringParam`	ASCII string

For each parameter the library function *ufoProcessParamDefine* (or *ufoProcessEnumParamDefine* for parameters of type *ufoEnumParam*) must be called to define its identification, type, and group.

Parameter groups should be defined with *ufoProcessParamGroupDefine* to give each group an identification.

Parameters of type *ufoIntegerParam, ufoFloatParam, ufoNormalisedParam, ufoRectangleParam, ufoPos3dParam* or *ufoScale3dParam,ufoPos2dParam* or *ufoScale2dParam,* should have their limits defined with *ufoProcessSetParamLimits*.

By default, parameters will be animatable, which means that as values are adjusted at different frames, key values will automatically be added. If a parameter value is intended to be a constant value over time, use *ufoProcessSetParamAnimAllow*.

Default values or keyframe animations should always be set for each parameter, for which there are the library functions *ufoProcessSetParamDefaultValue/s* or *ufoProcessSetParamDefaultKey/s*.

For transitional effects, if a parameter has default parameter keys which form a transition animation, then the *ufoProcessSetParamAutoTransition* library function can set the parameter keys to be automatically fitted to a transition length when first used.

By default, parameters are laid out in an automatically generated GUI. They are formatted from the top left of a rectangular area in columns. The function *ufoProcessSetParamColumnHint* allows you to give the application a hint about which column a parameter tool should be positioned in.

Certain types of parameters can be ganged together. The supported types are: *ufoIntegerParam*, *ufoFloatParam*, *ufoNormalisedParam,* and *ufoAngleParam*. Ganged parameters have an extra control to allow all the sliders to be moved simultaneously, such that when the gang control is switched on and any one ganged parameter is changed, then all ganged parameters are changed by the same amount. There are two functions you need to call when defining parameters in order for them to be ganged: *ufoProcessSetParamGang* can be called to indicate that a set of parameters (of the same type) should be grouped together with a gang control. *ufoProcessSetParamGangLabel* sets a collective title for the ganged parameters.

The *ufoProcessSetEffectType* function sets the type of effect group that the UFO belongs to. Currently this is only to inform whether the effect type is a transition or not, so the only two possible values are *ufoUndefinedEffect* or *ufoTransitionEffect*.

UFO Editor Definition

An external user interface to a UFO process is defined by supplying UFO editor user functions. These communicate with the application via the UFO interface library, primarily through functions declared in *ufoEditor.h*. An external user interface to a UFO process may contain a user-defined GUI, screen interaction and screen drawing into the RGB and Overlay image planes.

The definition and initialization of a UFO process editor must be supplied in the *ufoEditorCreate* user function. This includes the creation of any user-defined GUI which is to be used in the Editor and specification of which parameter groups the GUI and/or screen interaction will be provided for.

ufoEditorCreate returns a pointer to the base window of the GUI, which should be an X Motif widget. If the editor is only to contain screen drawing or screen interaction, then a null pointer is returned to indicate that no external GUI is provided.

Standard GUI parameter editors can be embedded within user-defined GUI editors, using the *ufoEditorCreateStandardEditor* library function. This creates a standard invocation of a parameter editor within a custom GUI, such as a slider for a *ufoFloatParam*, an option menu for a *ufoEnumParam*, a colour swatch for a *ufoColourRGBParam*, etc.

You may also embed ganged parameters within user-defined GUI editors, however, you only need to call *ufoEditorCreateStandardEditor* for the first parameter in the gang.

Each copy of a particular UFO process currently has the same editor instance associated with it. However, the implementation can always change, such that there could be more than one instance, or such that an editor may get destroyed and recreated. Therefore, all UFO editor user functions and most UFO editor library functions have an editor instance passed as the first argument.

Raster Inputs and Outputs

The number of raster inputs and outputs must be defined in the *ufoProcessDefine* user function. For each input or output, the library function *ufoProcessRasterInDefine* or *ufoProcessRasterOutDefine* must be called to define its identification and suggested working component combination. This can be RGB, RGB-Alpha, Alpha, or any.

If an input does not need to be connected for the effect to work, then *ufoProcessSetRasterInOptional* should be called to set it to optional. The default selection state for the optional input can be set with *ufoProcessSetRasterInOptionalDefault*.

UFO processes can support input and output rasters of nine different pixel types:

- 8-bits per component. Component range is 0 to 255.

Function	Description
`ufoRGB8PixelType`	RGB packed UBGR (U is redundant 32-bits per pixel)
`ufoA8PixelType`	Alpha 8-bits per pixel
`ufoRGBA8PixelType`	RGB-Alpha packed ABGR 32-bits per pixel

- 16-bits per component. Component range is 0 to 16383 (14 bits).

Function	Description
`ufoRGB16PixelType`	RGB packed BGR 48-bits per pixel
`ufoA16PixelType`	Alpha 16-bits per pixel
`ufoRGBA16PixelType`	RGB-Alpha packed ABGR 64-bits per pixel

- Floating point 32-bits per component. Component range is 0.0 to 1.0.

Function	Description
`ufoRGBFPixelType`	RGB packed BGR 96-bits per pixel
`ufoAFPixelType`	Alpha 32-bits per pixel
`ufoRGBAFPixelType`	RGB-Alpha packed ABGR 128-bits per pixel

By default, the rasters supplied to the inputs and outputs can be of any of the supported raster types. There are generic pixel access functions, described later, which can be used to set and get RGB, RGB-Alpha, or Alpha normalized (0.0 to 1.0) component values, generating sensible results regardless of a

raster's actual type. These functions are fine for many one-off or mathematically-intensive effects.

However, in the majority of cases, direct pixel access using pointers to pixel structures and using integer arithmetic will increase rendering performance, and may be necessary. In this case, the UFO code will probably only support a narrow subset of the entire possible combinations of input and output pixel types that may get connected to it. There are two ways to specify how input and output pixel type combinations may be converted to a combination that the UFO code supports. The recommended method is by supplying the *ufoProcessSpecifyConvertPixelTypes* user function. The pixel types of the rasters that will be supplied to the UFO process inputs and output are passed as arguments. If you would like the application to pre-convert the input rasters or supply a different type output to be converted after rendering, then the pixel type arguments should be changed.

Alternatively, although not recommended any more, the combinations of input and output pixel types supported by the UFO rendering code can be defined using *ufoProcessSetPixelTypeCombinations*. This allows multiple specifications of the desired pixel types for each input and output.

Some of the values used to describe desired pixel type values can indicate *no specific* pixel type or *any* pixel type of a *specific component type*. The full set of available values to specify a supported pixel type are:

- To specify any pixel type

`ufoAnyPixelType`	any pixel type

- To specify any pixel type of a component type

`ufoAny8PixelType`	any 8-bits per component pixel type
`ufoAny16PixelType`	any 16-bits per component pixel type
`ufoAnyFPixelType`	any floating point (32-bits per component) pixel type

- To specify an actual pixel type

```
ufoRGB8PixelType
ufoA8PixelType
ufoRGBA8PixelType
ufoRGB16PixelType
ufoA16PixelType
ufoRGBA16PixelType
ufoRGBFPixelType
ufoAFPixelType
ufoRGBAFPixelType
```

User Data

User data can be attached to a UFO process instance and to a UFO editor instance. This can be set and accessed within any user functions using *ufoProcessSetUserData, ufoProcessGetUserData, ufoEditorSetUserData*, and *ufoEditorGetUserData*, respectively.

If a UFO process instance is copied, then the associated process user data can be copied with a user-defined method by supplying the user function *ufoProcessCopyUserData*. Similarly, if a UFO process instance is destroyed (that is, put away) it can be tidied up with a user-defined method by supplying the *ufoProcessDeleteUserData* user function.

There are user functions to facilitate reading and writing of UFO user data from and to a process file on disk, along with the standard parameter and input and output settings of UFO process. The data may be saved and loaded in ASCII format by supplying the *ufoProcessWriteAsciiData* and *ufoProcessReadAsciiData* user functions, respectively. Alternatively, the data may be saved and loaded in binary format by supplying the user functions *ufoProcessWriteBinaryData* and *ufoProcessReadBinaryData*, respectively.

Currently, there is only one UFO editor instance in existence at any one time, therefore there are no corresponding *ufoEditorCopyUserData* and *ufoEditorDeleteUserData* user functions. If a UFO editor were to be destroyed, the editor user data can be tidied up with a user-defined method by supplying the *ufoProcessDeleteUserData* user function.

UFO writers should refrain from using global data in their UFO code for storing structures and values commonly used during rendering, and adopt a method which distinguishes between the different instances of a UFO using UFO user data. Essentially, this is to safeguard against multiple instances of the same UFO process being used simultaneously and accessing the same data. In particular, writing to global data during rendering is likely to cause errors if multiprocessing.

Rendering

Render User Functions

You can render in one of three ways: by rectangle, line, or pixel. The rectangle method (probably the most natural to most graphics programmers) uses the *ufoProcessRenderRectangle* user function which contains code to render a rectangular region of an output raster buffer. However, by default Media Illusion parallelizes your code for a multi-processor machine when the *ufoProcessRenderLine* or *ufoProcessRenderPixel* user function is defined. The simplest way of ensuring the performance increases (near) linearly with the number of processors available on a multi-processor machine is to supply one of the latter two functions.

Multi-threading

It is recommended that if you supply a *ufoProcessRenderLine* or *ufoProcessRenderPixel* function then you ensure that the function is either multi-process safe (MP Safe), or that you turn the automatic multi-threading off. Automatic multi-threading may be turned off completely by calling *ufoSetMPSafe* once only in the UFO definition function, or on a render pass basis by calling *ufoSetMPSafe* around the MP unsafe passes.

If you want to handle your own multi-threading, then it is recommended that use the application's multi-threading libraries and the set of utility functions for multi-threading. *ufoMPFork* creates extra threads that execute the supplied function argument in parallel with the calling thread. *ufoMPForkMyId* returns an integer within the range 0..n-1 when n is the number of threads running in parallel. The thread with ID 0 is always be the original parent thread. *ufoMPForkGetNumProcs* returns the number of threads that are running in parallel.

Finally, *ufoMPPotentialNumProcs* may be used to query the maximum number of potential threads that are created by a call to *ufoMPFork*. This may be restricted by the number of processors on the host machine, multi-processing licensing and application command line arguments that restrict the number of threads to use.

Multiple Passes

As some rendering methods require multiple scans of the inputs and outputs, rendering can be split into multiple passes. The number of passes required can be set within *ufoProcessDefine* or any user function before the start of the first pass of a render at a particular frame using *ufoProcessSetNumberRenderPasses*. Within the render user functions the current render pass can be inquired with *ufoProcessGetRenderPass*.

Line rendering can also work vertically to allow for multiple pass methods which require vertical scanning of particular passes using *ufoProcessSetRenderPassMode*.

Render Slicing and Checking for Abort

If rendering is slow, output regions are sliced or tiled into smaller regions by the application, so that it can check for a user interrupt (for example, pressing the Stop button). The region size is chosen, so that rendering is performed in bursts of around 2 seconds in between which it can be halted. If this does not suit the rendering algorithm used, then it can be prevented by calling *ufoProcessSetTilingAllow*.

If you want to take complete control of rectangle rendering and do not want the application to automatically slice your render, but you do want to check for the user interrupting the current render, then the function *ufoProcessCheckAbort* can be called intermittently within the rendering code. If it returns a value of 1 then the render should terminate as soon as possible. For display purposes the function also takes a *percentage_done* argument. If you are performing multi-threading then this should be called with the original thread that the render function was called with.

Unchanged Inputs

Often during editing, subsequent renders at the same frame may have input rasters which have not changed since the previous render. Many rendering techniques may be able to take advantage of this, probably in conjunction with certain parameters being the same, by caching some intermediary result. *ufoProcessIsInputDirty* in conjunction with *ufoProcessResetRasterInDirty* can be called for each input to determine this.

Render Mode and Limits

ufoProcessGetRenderMode returns whether the render is a single frame or sequence and whether it is part of an interactive edit or preview sequence or a fully rendered sequence. This is useful for ignoring parameters in a full render which control accelerated methods used in editing or previewing.

The functions *ufoProcessGetMinEffectLimit* and *ufoProcessGetMaxEffectLimit* can be used to return current render effect start and end frame limits.

Background rendering

The function *ufoCheckInBackground* checks if the main application is running in foreground mode (with interactive user interface) or in background mode (with no user interface). This may be useful for licensing purposes.

Field Rendering

The functions *ufoProcessFieldRendering* and *ufoProcessWhichField* allow you to inquire whether a current render call is a frame or a field render, and if it is a field rendering then which field.

Aspect Ratio Correction

This function *ufoProcessGetAspectRatioCorrection* applies the aspect ratio correction factor during rendering. The value that is returned is the scale that will apply to the x pixel dimension of any images in order to display them with the correct aspect.

For example, if a PAL 720x576 image is to be displayed at 16:9, then the aspect ratio correction returned will be (576/720)*(16/9) = 1.42222. The actual calculation is more complex than this for video.

Any effects involving transformations, such as rotation will need to take this aspect ratio into account in order to appear correct when displayed in the final output aspect ratio.

Pre- and Post-Render

There are three pairs of user functions which, if supplied, are called before and after rendering a sequence of frames, a frame and a pass, respectively. These can be used to set up commonly used data, allocate, and free temporary rasters and memory buffers and so on. *ufoProcessPreSequenceRender* and *ufoProcessPostSequenceRender* are called at either end of a sequence of renders. *ufoProcessPreRender* and *ufoProcessPostRender* are called before and after rendering at a frame. *ufoProcessPreRenderPass* and *ufoProcessPostRenderPass* are called before and after each pass of rendering at a frame.

In addition, you can supply the user function *ufoProcessCanAvoidRendering*. This can be used to determine whether processing of the UFO can be avoided because the current parameter setup has no effect. In this event, one of the UFO's inputs can be used in place of its output. Therefore, if processing can be avoided, then the function takes a parameter to be set to the input index that the UFO has no effect on. This replaces a previously unused UFO function *ufoProcessHasNoEffect*.

Parameter Access

Within the render, user functions and certain pre- and post-render user function parameters may be accessed for their values. *ufoProcessGetParamValue/s* can be called to get the current value(s) of a parameter, *ufoProcessGetParamValueAtTime/s* can be called to get the value(s) of a parameter at a specific time (*ufoProcessGetTime* can be used to get the current frame time of rendering.) These functions return double-precision floating-point values, so conversion to integer is required for parameters of type *ufoBooleanParam, ufoEnumParam*, and *ufoIntegerParam*.

You can determine if a parameter has been edited (through some user interface event) by calling *ufoProcessIsParamSet*.

The *ufoProcessParamsEdited* function is called whenever any parameters are edited by the user. It supplies the list of parameter indices for the parameters that have been changed. The user can then react to the edit by changing additional parameter values. This function is useful, for example, for controlling presets through a *ufoEnumParam* parameter, which set a number

of other parameters, or for example, if you have a set of parameters which are alternative representations of a single value.

Raster Access

Within the render user functions, editor user functions, and certain pre- and post-render user functions, you can access rasters for reading or writing.

ufoProcessGetRasterIn returns a handle to an input raster and *ufoProcessGetRasterOut* returns a handle to an output raster. *ufoEditorGetRasterViewed* returns a handle to the viewed raster during editing.

Given a raster handle, there are a suite of UFO library functions for interacting with the raster (see *ufoRaster.h*). All returned raster handles should be tested against null as, under certain circumstances, such as prior to processing or getting from unconnected UFO process inputs, the raster may not be available.

All the currently supported raster types are rectangular arrays of pixels, with pixel components (RGB, Alpha, RGB-Alpha) packed together. The raster interface takes account of an arbitrary base offset, width and height associated with the raster's pixel array.

ufoRasterGetLimits, ufoRasterGetWidth, and *ufoRasterGetHeight* return information on a raster's rectangular geometry.

ufoRasterGetPixelType, ufoRasterGetPixelSize, and *ufoRasterComponentSize* return information on the pixel type.

ufoRasterGetPixelAddress(x,y) returns the pointer to the pixel at (X,Y). *ufoGetBufferAddress* returns the base address of the raster's buffer (not necessarily at (0,0)). *ufoRasterGetBufferSize* returns the full size of the raster's buffer (in bytes).

ufoRasterGetPixelCol, ufoRasterGetPixelColAlpha, and *ufoRasterGetPixelAlpha* return normalized (0.0..1.0) component values at a pixel. *ufoRasterSetPixelCol, ufoRasterSetPixelColAlpha,* and *ufoRasterSetPixelAlpha* set normalized component values at a pixel. These functions work on all types of rasters regardless of their actual type, performing sensible conversions as necessary (detailed in *ufoRaster.h* and the appendices). These functions do not perform checks for the pixel coordinates that are outside the raster, so when writing UFO effects which override automatic clamping on certain inputs (see *ufoProcessSetRasterInEdgeMode*) or any effect which reads input pixels at coordinates which are offset from a pixel being rendered to output, you should perform bounds checks.

ufoRasterCopy and *ufoRasterCopyPart* copy whole and parts of rasters from one to another, handling pixel conversions as necessary. *ufoRasterAlloc* allocates a temporary raster of a specified pixel type. *ufoRasterAllocCopy* and *ufoRasterAllocCopyPart* allocate a copy of whole or part of an existing raster and optionally copy the pixel data from the original to the new. *ufoRasterFree* frees any raster previously allocated through the UFO interface library. You

should use these functions for raster allocation and freeing, as they make use of the Media Illusion memory manager.

Temporal Frame Raster Access

Some types of effects are classified as inter frame, such that for each output at a current frame they need to read a frame or frames from an input which are not at the current frame. For instance, a retiming effect or motion blur effect.

ufoProcessGetRasterInAtFrame returns the raster for an input at a specified frame. The function includes a field argument which is used when field rendering (see *ufoProcessFieldRendering*). Each call to *ufoProcessGetRasterInAtFrame* with a unique frame (and field) will lock a raster in memory by the Media Illusion memory manager until *ufoProcessReleaseRasterInAtFrame* is subsequently called with that frame. It is very important to use this function to unlock memory from the memory manager and not keep too many frames locked in memory simultaneously. If you don't call *ufoProcessReleaseRasterInAtFrame* then all accessed frame rasters will remain locked until after the UFO process has finished rendering at that frame. Currently, temporal access only works if the input is connected directly to a clip. *ufoProcessIsRasterInTemporal* can be used to check this. The function *ufoProcessGetRasterInTemporalLimits*() returns the frame limits available for temporal access on an input.

Scaled and Cropped Rendering

Media Illusion uses two principal techniques for accelerating the performance of rendering for interactive updating of effects during editing:

- *Scaled* rendering, where rendering can be performed at an arbitrary reduced resolution. The render method must take into account scaling of any geometric parameters by the render scale factor. The UFO library function *ufoProcessGetRenderScale* gets the X and Y scale factor. This technique trades variable amounts of precision against speed.
- *Cropped* rendering, where a rectangular subregion of the whole region to be rendered is selected. Cropping may also occur explicitly as the result of using a Crop process or transformation process, or implicitly through render-economizing techniques employed by the Media Illusion hierarchy traversal method.

To obtain the full render resolution, *ufoProcessGetFullRenderRectangle* returns the full rectangle resolution of the final target clip in the process hierarchy being rendered, regardless of any render cropping, scaling or use of reduced resolution material. This is useful for any effect which is based on a position or region relative to the full render region, such as a swirl around the centre or an explode from the centre.

Region Calculation

By default, an input has its edge mode set to *ufoEdgeClamp* such that an output render region will automatically be clamped to the limits of the input. However, UFO effects whose output render region may include pixels outside the input will need to call *ufoProcessSetRasterInEdgeMode* to set the edge

mode to *ufoEdgeNoClamp* and then perform their own limits checks on the input region when rendering. Effects that would use this include scaling down, and compositing of smaller images onto larger backgrounds.

When you render a hierarchy of processes of arbitrary complexity, Media Illusion uses cropping techniques to minimize the regions required to be rendered. By default, Media Illusion assumes that there is a one-to-one mapping between input pixels and output pixels. Effects, such as composites and colour space manipulations, perform calculations on pixel values at the same position in the inputs as the pixel being rendered in the output.

The defined region of a process' output is the bounding rectangle of the effect it can produce in an output raster given the bounding rectangle regions supplied to its inputs. Effects such as geometric transformations (scales, rotations, perspective, warps, etc.) can define output regions which are larger, smaller, or offset from those defined on its inputs. If a UFO process can do this, then you should supply the *ufoProcessCalcDefinedRectangle* user function. This is called before a frame is rendered and after all the process parameters have been set with their current values. The function call supplies the defined regions of each input and should return the defined region for all outputs. If this function is not supplied, the output defined region is assumed to be the same as the first input defined region.

The needed region of a process's input is the maximum bounding rectangle it needs to be supplied in an input in order to produce the required effect within the supplied output region. Effects such as convolutions which perform calculations on a number of input pixels in a vicinity around the corresponding output pixel, need input regions which are larger than the supplied needed output region by half the size of the convolution kernel in each direction. Effects, such as geometric transformations, need an input region which is the inverse transformation of the supplied needed output region.

If a UFO process is required to render some arbitrary rectangular region on its outputs, and in turn it pre-requires rectangular regions on its inputs to be rendered which may be greater, smaller and/or offset from that output region, then you should supply the *ufoProcessCalcNeedRectangles* user function. This is called before a frame is rendered and after all the process parameters have been set with their current values. The function call supplies the needed region of output and should return the needed region for all inputs. If this function is not supplied, the input needed regions are assumed to be the same as the output needed region.

With both region calculation user functions, the supplied rectangle regions do not take into account any render scaling and the returned rectangle regions should not either.

It is important that you understand the requirement of these functions if you want your UFO process to behave properly when there is any cropping (automatically or manually) during rendering.

Screen Drawing and Screen Interaction

You can provide screen drawing through the *UFOEditorRGBDraw* and *UFOEditorOverlayDraw* user functions. They let the user draw OpenGL graphics in the RGB image plane and the overlay image plane respectively.

If provided, each function will be called by the UFO editor during editing, when the draw window needs updating or a parameter has changed. You should use the *ufoEditorGetOverlayIndex* library function within *ufoEditorOverlayDraw* to get a correct colour index to be used when setting overlay colour indices in OpenGL.

We recommend that you supply *ufoEditorOverlayDraw* in interactive editors as drawing into the overlay image plane is more efficient than drawing into the RGB image plane.

You can provide screen interaction using the following user functions; any subset of which may be combined to produce the desired interface. *ufoEditorMouseDown*, *ufoEditorMouseUp*, *ufoEditorMouseDrag*, and *ufoEditorMousePosition* are called on the corresponding mouse events and *ufoEditorKeyEvent* is called on a keyboard key press. The *ufoEditorDraw* library function causes the editor to set up context for OpenGL drawing, and calls *ufoEditorOverlayDraw* or *ufoEditorRGBDraw*. This speeds up screen drawing during screen or GUI interaction.

OpenGL projections are set up for projection of the full or master raster resolution coordinate space on the screen. For mouse interaction, the supplied X,Y position will be in full or master raster resolution pixel coordinates.

Media Illusion lets you zoom the viewed raster during editing by an arbitrary factor. The *ufoEditorRGBDraw* and *ufoEditorOverlayDraw* user functions may need to take into account any raster zooming. The UFO library function *ufoEditorGetPixelSize* gets the X and Y pixel size of the viewed raster indicating the zoom factor.

Memory Allocation

You should use the UFO memory allocation mechanism to dynamically allocate and free memory. This will make use of the Media Illusion memory manager. The functions *ufoMalloc, ufoRealloc,* and *ufoFree* have the same functionality as the *malloc, realloc* and *free system* calls.

Order of UFO Process User Function Calls

UFO user functions are called as follows:

Media Illusion Initialization

```
ufoProcessDefine
```

UFO Process Instance Copy

```
ufoProcessCopyUserData
```

UFO Process Instance Destruction

```
ufoProcessDeleteUserData
```

UFO Process Instance Saving and Loading

```
ufoProcessWriteAsciiData
ufoProcessReadAsciiData
ufoProcessWriteBinaryData
ufoProcessReadBinaryData
```

Rendering Sequence

```
ufoProcessPreSequenceRender
       :
    Rendering Frame f
        ufoProcessCanAvoidRendering
        ufoProcessCalcDefinedRectangle
        ufoProcessCalcDefinePixelType
        ufoProcessCalcNeedRectangles
        ufoProcessSpecifyConvertPixelTypes
        ufoProcessPreRender
        Render Pass 0
            :
        Rendering Pass p
          ufoProcessPreRenderPass
          ufoProcessRenderRectangle
          [ufoProcessRenderRectangle ..]
```

(possibly rendered in tiles or slices)

or

```
ufoProcessRenderLine ..
[ufoProcessRenderLine ..]
```

or

```
ufoProcessRenderPixel..
[ufoProcessRenderPixel..]
ufoProcessPostRenderPass
Rendering Pass p+1
:
ufoProcessPostRender
Render Frame f+1
:
ufoProcessPostSequenceRender
```

Compiling and Installing UFOs

UFO code should be compiled and linked into a dynamic shared object (DSO). The DSO file name must have the extension *.ufo* and must be copied to the directory */var/avid/parallax/parallax_process_ufos* (/var/avid/parallax/parallax_process_ufos_n32 for Media Illusion version 6.0v1 and above) or a subdirectory of that directory.

If a UFO is found in this directory, it will appear in the UFO section of the process selection menu. If a UFO is found in a subdirectory of this directory it will appear under a category with the name of the subdirectory. These can be existing categories or new categories. For example, a simple *makefile* to build and install some UFOs might be as follows:

```
.SUFFIXES:.c .o .ufo
UFO_SRCS = mix.c \
     colour_mix.c \
     swirl.c
UFO_TARGETS = $(UFO_SRCS:.c=.ufo)
UFO_DIR = /var/avid/parallax/parallax_process_ufos
$(UFO_TARGETS)::
     cc -c -g $(@:.ufo=.c)
     ld -shared -exports_file export_symbols -o $@
$(@:.ufo=.o)
all:
     make $(UFO_TARGETS)
release:
     - mkdir $(UFO_DIR)
     cp $(UFO_TARGETS) $(UFO_DIR)
```

The *-exports_file export_symbols* flag prevents two (or more) UFOs with global functions of the same name from calling each other's functions instead of their own functions. *export_symbols* is a file containing the symbols for all the UFO user functions. When linked with this flag, all global symbols are hidden apart from the user function global symbols in this file.

A more complex makefile which can handle C++ and C source, example UFO source code, and the *export_symbols* file is provided as part of the UFO release. The example UFOs can be built and installed using this makefile.

UFO User Functions

ufoProcessDefine

```
ufoProcess ufoProcessDefine(void);
```

Must be supplied to provide the definition of a UFO process. The process instance handle returned from the *ufoProcessCreate* function call should be returned from this function.

ufoProcessCopyUserData

```
void* ufoProcessCopyUserData(
    ufoProcess      process_instance,
    void            *user_data);
```

Refer to the following table for command descriptions:

Command	Description
`process_instance`	original process instance
`user_data`	user data attached to original process instance

Returns pointer copied user data. The *ufoProcessCopyUserData* function should be supplied if user data is set in any UFO user function with *ufoProcessSetUserData*, and requires a user-defined copy method to be used whenever process instances are copied. The function should return the pointer to the copied user data.

ufoProcessDeleteUserData

```
void ufoProcessDeleteUserData(
    ufoProcess      process_instance,
    void            *user_data);
```

ufoProcessDeleteUserData should be supplied if user data is set in any UFO user function with *ufoProcessSetUserData*, and requires a user-defined destruction method to be used whenever process instances are destroyed.

ufoProcessWriteAsciiData

```
char* ufoProcessWriteAsciiData(
ufoProcess process_instance);
```

This function is optional. If supplied, it will be called just before the UFO process is about to be saved by the application to a descriptor file. If the UFO holds private user data, which is required to persist after the application is terminated, then the function can return an ascii data string representing that

data. The string is then saved within the descriptor file. If no data is required to be saved, the function should return a null pointer.

The stored data string must contain no quotes and be null terminated.

ufoProcessReadAsciiData

```
void ufoProcessReadAsciiData(
ufoProcess process_instance,
char *ascii_data_string);
```

This function is optional. If supplied, it will be called just after the UFO process has been loaded by the application from a descriptor file. The ASCII string will contain the exact representation of the private data which was saved when the UFO process was saved to disk, as returned by *ufoProcessWriteAsciiData*.

The contents of the string, *ascii_data_string*, should copied if required beyond the duration of the function call.

ufoProcessWriteBinaryData

```
void ufoProcessWriteBinaryData(
ufoProcess process_instance,
size_t *data_length,
void **data_buffer,
int *free_data_buffer);
```

Refer to the following table for command descriptions:

Command	Description
`data_length`	returns the byte length of the data buffer returned
`data_buffer`	returns the data buffer
`free_data_buffer`	return 1: application should free the data buffer returned (with free), 0: application should not free the data buffer returned

This function is optional. If supplied, it will be called just before the UFO process is about to be saved by the application to a descriptor file. If the UFO holds private user data which is required to persist after the application is terminated then the function can return a binary data buffer representing that data. The data buffer is then saved within the descriptor file. If no data is required to be saved, the function should return a value of 0 in the data_length parameter and a null pointer in the data_buffer parameter.

ufoProcessReadBinaryData

```
void ufoProcessReadBinaryData(
ufoProcess process_instance,
```

```
size_t data_length,
void *data_buffer);
```

Refer to the following table for command descriptions:

Command	Description
`data_length`	the byte length of the data buffer supplied
`data_buffer`	supplied data buffer

This function is optional. If supplied, it will be called just after the UFO process has been loaded by the application from a descriptor file. The data buffer will contain the exact representation of the private data which was saved when the UFO process was saved to disk, as returned by *ufoProcessWriteAsciiData*.

The contents of the data buffer should copied if required beyond the duration of the function call.

ufoProcessPreSequenceRender

```
void ufoProcessPreSequenceRender(
    ufoProcess      process_instance);
```

This function is optional. If supplied, it is called before a sequence of frames is rendered. *ufoProcessPreSequenceRender* should be supplied if specific initializations or data allocations need to be performed before a sequence of renders.

ufoProcessPostSequenceRender

```
void ufoProcessPostSequenceRender(
    ufoProcess      process_instance);
```

This function is optional. If supplied, it is called after a sequence of frames is rendered. *ufoProcessPostSequenceRender* should be supplied if specific de-allocations or tidying of data need to be performed after a sequence of renders.

ufoProcessCanAvoidRendering

```
int ufoProcessCanAvoidRendering(
ufoProcess process_instance,
int *no_effect_input_index);
```

Refer to the following table for command descriptions:

Command	Description
`no_effect_input_index`	input to copy to the output when avoidance can be done

Returns 1: avoidance can be performed, 0: avoidance cannot be performed.

This function is optional. If supplied, it will be called before the rendering any rectangular region of the UFO process outputs. When called, the current values of all parameters can be inquired and used to determine whether the process will have any effect. If processing would have no effect and can be avoided then 1 should be returned and the *no_effect_input_index* parameter should be set to the input index which the UFO has no effect on. Otherwise 0 should be returned. This replaces a previously unused UFO function *ufoProcessHasNoEffect*().

ufoProcessCalcDefinedPixelType

```
void ufoProcessCalcDefinedPixelType(
ufoProcess process_instance,
ufoPixelType* input_pixel_types,
ufoPixelType* preferred_output_pixel_type);
```

Refer to the following table for command descriptions:

Command	Description
`input_pixel_types`	array of pixel types of rasters to be supplied to inputs
`preferred_output_pixel_type`	preferred output pixel type to be returned

This function is optional. If supplied, it will be called before the rendering any rectangular region of the UFO process outputs, and periodically during user interface interactivity. Given the determined pixel types of the input rasters the function should calculate and return the preferred defined output pixel type to which it can render.

This function is also called periodically during tree editing in order to colour code the input and output connection lines of the tree line UI. In this case there may be inputs which are not connected. If this happens then the corresponding input_pixel_types element will be set to *ufoUndefinedPixelType*.

If an input is optional, then you can test for whether it is selected using *ufoProcessIsRasterInDefined*.

If not supplied, the output *defined* pixel type is calculated using the application default method.

This function should be supplied where application default method does not automatically generate the preferred output pixel type or where the output pixel type may vary depending on the state of the parameters in the process.

ufoProcessSpecifyConvertPixelTypes

```
void ufoProcessSpecifyConvertPixelTypes(
ufoProcess process_instance,
ufoPixelType* input_convert_pixel_types,
ufoPixelType* output_convert_pixel_type);
```

Refer to the following table for command descriptions:

Command	Description
`input_convert_pixel_types`	array of input pixel type pre conversions, to be returned
`output_convert_pixel_type`	output pixel type post conversion, to be returned

This function is optional. If supplied, it will be called before the rendering of any rectangular region of the UFO process outputs.

The pixel types of the rasters that will be supplied to the UFO process inputs and output are passed in the input_convert_pixel_types array and output_convert_pixel_type variable, respectively. If you would like the application to pre-convert the input rasters or supply a different type output to be converted after rendering, then change these values.

This function is intended as a better alternative to the *ufoProcessSetPixelTypeCombinations* function library function.

ufoProcessPreRender

```
void ufoProcessPreRender(
    ufoProcess      process_instance,
    int             x1,
    int             y1,
    int             x2,
    int             y2);
```

Refer to the following table for command descriptions:

Command	Description
`x1,y1,x2,y2`	full region to be rendered

This function is optional. If supplied, it is called directly before the rendering of a rectangular region of the UFO process outputs.

When called, all raster inputs and outputs and parameters will be set in the process, such that all their attributes and values can be inquired. *ufoProcessPreRender* should be supplied if specific data initializations or allocations need to be performed before a render.

If the *ufoProcessRenderLine* or *ufoProcessRenderPixel* render function is supplied, performance will improve if parameter values are inquired and copied into some representation in the UFO user data within this function, and then referenced from the user data during rendering.

ufoProcessPostRender

```
void ufoProcessPostRender(
    ufoProcess      process_instance);
```

This function is optional. If supplied, it is called every time directly after the rendering any rectangular region of the UFO process outputs. *ufoProcessPostRender* should be supplied if specific de-allocations of data need to be performed after a render.

ufoProcessPreRenderPass

```
void ufoProcessPreRenderPass(
    ufoProcess      process_instance,
    int             pass_number);
```

This function is optional. If supplied, it is called directly before each pass of rendering of any rectangular region of the UFO process outputs. *ufoProcessPreRenderPass* should be supplied if specific data initializations or allocations need to be performed before a particular rendering pass, where multiple rendering passes are required (see *ufoProcessSetNumRenderPasses*).

ufoProcessPostRenderPass

```
void ufoProcessPostRenderPass(
    ufoProcess      process_instance,
    int             pass_number);
```

This function is optional. If supplied, it is called directly after each pass of rendering of any rectangular region of the UFO process outputs. *ufoProcessPostRenderPass* should be supplied if specific data de-allocations need to be performed after a particular rendering pass, where multiple rendering passes are required (see *ufoProcessSetNumRenderPasses*).

ufoProcessRenderPixel

```
void ufoProcessRenderPixel(
    ufoProcess      process_instance,
    int             x,
    int             y);
```

ufoProcessRenderLine

```
void ufoProcessRenderLine(
    ufoProcess      process_instance,
```

```
    int             x1,
    int             x2,
    int             y);
```

ufoProcessRenderRectangle

```
void ufoProcessRenderRectangle(
    ufoProcess      process_instance,
    int             x1,
    int             y1,
    int             x2,
    int             y2);
```

One of these three functions must be supplied. It calls every pixel, line or rectangle, respectively, within a single render pass. On the last render pass, (which is the only render pass by default–unless multiple passes have been set), the region supplied in the arguments should be rendered to the UFO process outputs.

If the current render pass mode has been set to *ufoPassVertical* using *ufoProcessSetRenderPassMode* then the argument x and y values are swapped.

ufoProcessCalcDefinedRectangle

```
void ufoProcessCalcDefinedRectangle(
    ufoProcess        process_instance,
    ufoRectangle*     input_rectangles,
    ufoRectangle*     defined_output_rectangle);
```

Refer to the following table for command descriptions:

Command	Description
`input_rectangles`	array of defined input rectangle regions
`defined_output_rectangle`	defined rectangle region to be returned

This function is optional. If supplied, it is called before the rendering of the process at a frame. The function should calculate and return the defined region for all outputs from the defined regions supplied for each input.

The supplied rectangle regions do not take into account the current render scaling, and the returned rectangle region should not either.

If not supplied, the output defined rectangle region defaults to the same as the first input.

The supplied input_rectangles regions do not take into account the current render scaling, and the returned defined_output_rectangle region should not either.

ufoProcessCalcNeedRectangles

```
void ufoProcessCalcNeedRectangles(
    ufoProcess        process_instance,
    ufoRectangle*     output_rectangle,
    ufoRectangle*     needed_input_rectangles);
```

Refer to the following table for command descriptions:

Command	Description
`output_rectangle`	needed rectangle region
`needed_input_rectangles`	array of needed input rectangle regions to be returned

This function is optional. If supplied, it is called before a process is rendered at a frame. Given the output region to be rendered, the function should calculate the required needed regions, which are the (sensible) maximum sized rectangle regions of input which are required in order to render to that output region properly.

The supplied *needed_input_rectangle* array is already pre-set to the maximum defined rectangle. This is useful if you cannot determine a finite needed region, as you can clip to it or leave it as it is.

The supplied *output_rectangle* region does not take into account the current render scaling, and the returned *needed_input_rectangles* regions should not either.

If *ufoProcessCalcNeedRectangles* is not supplied, the input needed regions default to the same as the output.

ufoProcessParamsEdited

```
void ufoProcessParamsEdited(
ufoProcess process_instance,
int number_params_edited,
int *param_index_array,
int *number_additional_changed_params,
int *additional_changed_param_index_array);
```

Refer to the following table for command descriptions:

Command	Description
`number_params_edited`	the number of parameter which have been edited
`param_index_array`	array of indices of the parameters which have been changed
`number_additional_changed_params`	return number of additional parameters that have been changed
`additional_changed_param_index_array`	return array of indices of parameters that have been changed

This function is optional. If supplied, it is called whenever any UFO parameters are changed by a user interface tool, and supplies the list of parameters that have been changed. The user can then act on the edit by changing additional parameters. The list of additional parameters changed should be returned, so that undo and redo can be performed properly, and so that other application GUIs can be updated properly.

This function is useful, for example, for controlling presets on a set of parameters by a *ufoEnumParam* parameter.

This function is called on parameter changes made by any application supplied parameter editor, including changes made by any ufo custom editor (as specified in the *ufoEditorNotifyEdited* library function.) This function is not called on undo, redo, reset, or load operations.

ufoEditorCreate

```
void* ufoEditorCreate(
    ufoEditor       editor_instance,
    void*           parent_window,
    int*            screen_interact_groups,
    int*            gui_groups);
```

Refer to the following table for command descriptions:

Command	Description
`parent_window`	X Motif widget
`screen_interact_groups`	array of parameter groups the screen interaction handles
`gui_groups`	array of parameter groups the GUI handles

This function is optional. If supplied, *ufoEditorCreate* is called the first time an instance of the UFO process is edited. Given the parent window, any GUI for

the UFO process interface should be built, and its base window returned. The window system uses X Motif, so the *void** window type should be cast to the X widget type: *Widget*.

If only screen drawing or screen interaction is required instead of GUI interaction, *ufoEditorCreate* should return null.

The *screen_interact_groups* array is set to 1 for each UFO parameter group. If screen interaction is not needed for a particular parameter group, then the respective element for the group in the array should be set to 0.

The *gui_groups* array is set to 1 for each UFO parameter group. If the created GUI is not needed for a particular parameter group, then the respective element for the group in the array should be set to 0.

ufoEditorDelete

```
void ufoEditorDelete(
    ufoEditor       editor_instance);
```

You should supply *ufoEditorDelete* if editor user data is set in any UFO user function (with *ufoEditorSetUserData*), or user structures are created within the editor which require a user-defined destruction method to be used whenever editor instances are destroyed.

ufoEditorSetParamGroup

```
void ufoEditorSetParamGroup(
    ufoEditor       editor_instance,
    ufoProcess      process_instance,
    int             parameter_group);
```

Refer to the following table for command descriptions:

Command	Description
`parameter_group`	index (0..number_parameter_groups) of parameter group

This function is optional. If supplied, it is called whenever a UFO process is edited and whenever the parameter group is changed. If the parameter group change affects the editor GUI, it should be implemented within this function.

ufoEditorHideGui

```
void ufoEditorHideGui(
    ufoEditor       editor_instance);
```

This function is optional. If supplied, *ufoEditorHideGui* will be called whenever the UFO Process is required to hide the editor GUI.

ufoEditorHideGUI should be supplied if the editor GUI contains pop-up dialogs.

ufoEditorKeyEvent

```
void ufoEditorKeyEnter(
    ufoEditor       editor_instance,
    ufoProcess      process_instance,
    void*           event);
```

Refer to the following table for command descriptions:

Command	Description
`event`	X Motif Event

This function is optional. If supplied, it will be called whenever a key event occurs during editing (when a key or combination of keys is pressed).

ufoEditorMouseDown

```
void ufoEditorMouseDown(
        ufoEditor       editor_instance,
        ufoProcess      process_instance,
        double          x,
        double          y,
        void*           event);
```

Refer to the following table for command descriptions:

Command	Description
`x,y`	mouse position in raster pixels
`event`	X Motif Event

This function is optional. If supplied, it will be called whenever a mouse button is pressed within the viewer during editing. The supplied X,Y position will be in full/master raster resolution pixel coordinates.

ufoEditorMouseDrag

```
void ufoEditorMouseDrag(
    ufoEditor       editor_instance,
    ufoProcess      process_instance,
    double          x,
    double          y,
```

```
    void*           event);
```

Refer to the following table for command descriptions:

Command	Description
`x,y`	mouse position in raster pixels
`event`	X Motif Event

This function is optional. If supplied, it will be called whenever a mouse is moved with a button pressed within the viewer during editing. The supplied X,Y position will be in full/master raster resolution pixel coordinates.

ufoEditorMousePosition

```
void ufoEditorMousePosition(
    ufoEditor       editor_instance,
    ufoProcess      process_instance,
    double          x,
    double          y,
    void*           event);
```

Refer to the following table for command descriptions:

Command	Description
`x,y`	mouse position in raster pixels
`event`	X Motif Event

This function is optional. If supplied, it will be called whenever a mouse is moved without a button being pressed, within the viewer during editing. The supplied X,Y position will be in full/master raster resolution pixel coordinates.

ufoEditorMouseUp

```
void ufoEditorMouseUp(
    ufoEditor       editor_instance,
    ufoProcess      process_instance,
    double          x,
    double          y,
    void*           event);
```

Refer to the following table for command descriptions:

Command	Description
`x,y`	mouse position in raster pixels
`event`	X Motif Event

This function is optional. If supplied, it will be called whenever a mouse button is released within the viewer during editing. The supplied X,Y position will be in full/master raster resolution pixel coordinates.

ufoEditorOverlayDraw

```
void ufoEditorOverlayDraw(
    ufoEditor       editor_instance,
    ufoProcess      process_instance);
```

This function is optional. If supplied, it will be called during editing and rendering when the viewer window needs updating or a process parameter has changed.

Any graphics drawn in the Open GL overlay plane during editing should be performed within this function. OpenGL projections are set up for projection of the full/master raster resolution coordinate space on the screen.

The UFO library function *ufoEditorGetOverlayIndex* gets the correct colour index to be used for setting the overlay draw colour in OpenGL. This is necessary as Media Illusion uses part of the overlay plane for other specific tools.

ufoEditorRGBDraw

```
void ufoEditorRGBDraw(
    ufoEditor       editor_instance,
    ufoProcess      process_instance);
```

This function is optional. If supplied, it will be called during editing and rendering when the viewer window needs updating or a process parameter has changed.

Any graphics drawn in the Open GL RGB Plane during editing should be performed within this function. OpenGL projections are set up for projection of the full/master raster resolution coordinate space on the screen.

ufoEditorShowGui

```
void ufoEditorShowGui(
    ufoEditor       editor_instance);
```

This function is optional. If supplied it will be called whenever the UFO process is required to show the editor GUI.

You should supply *ufoEditorShowGui* if the editor GUI contains pop-up dialogs.

ufoEditorUpdateGui

```
void ufoEditorUpdateGui(
    ufoEditor       editor_instance,
    ufoProcess      process_instance);
```

ufoEditorUpdateGui should be supplied if the UFO process has a defined editor GUI. This ensures that the GUI represents up-to-date parameter values of the currently edited UFO process.

If supplied, *ufoEditorUpdateGui* is called whenever the editor GUI interface needs to be redrawn.

UFO Library Functions

ufoProcessCreate

```
ufoProcess ufoProcessCreate(
    char            *name,
    int             number_inputs,
    int             number_outputs,
    int             number_parameters,
    int             number_parameter_groups);
```

Refer to the following table for command descriptions:

Command	Description
`name`	user interface name for process
`number_inputs`	number of image/mask/RGBA inputs
`number_outputs`	number of image/mask/RGBA outputs
`number_parameters`	number of parameters
`number_parameter_groups`	number of groups the parameters are divided into

ufoProcessCreate creates the initial UFO process definition. All used instances of this process are copied from this one (or copies of copies of this one).

ufoProcessCreate must be called within *ufoProcessDefine* to get an initial process instance handle to pass to all subsequent functions as required.

ufoProcessSetEffectType

```
void ufoProcessSetEffectType(
ufoProcess process_instance,
ufoEffectType effect_type);
```

This function may be called to set type of the effect group that the UFO belongs to. Currently the only setting other than the default, *ufoUndefinedEffect*, is *ufoTransitionEffect*. This may be set in order to inform the application that a UFO is a transition effect.

Within the application's Composition Editor, only transition effects may be used for performing transitions between track segments.

If you want parameter keyframes to automatically scale in duration during certain invocations in the application then you should use *ufoProcessSetParamAutoTransition* passing the specific parameter index.

By default the process effect type is *ufoUndefinedEffect*.

ufoProcessSetMPSafe

```
void ufoProcessSetMPSafe(
     ufoProcess      process_instance,
     int             mp_safe);
```

Refer to the following table for command descriptions:

Command	Description
`mp_safe`	1: following renders are MP safe (default), 0: not MP safe

ufoProcessSetMPSafe sets whether the following render is multi-processor safe (MP Safe) or not.

By default, MPSafe is true, and on multi-processor machines (such as SGI Onyx) UFO line and pixel render routines will be called in parallel. If your render routine or a pass of your render routine, is not MP safe then you should call this routine with *mp_safe* set to 0 for the period in which it is MP unsafe.

ufoProcessSetMPSafe may be called within *ufoProcessDefine* to set for all renders, within *ufoProcessPreRender* to set before each frame, or within *ufoProcessPreRenderPass* to set before each render pass.

ufoProcessSetNumberRenderPasses

```
void ufoProcessSetNumberRenderPasses(
     ufoProcess      process_instance,
     int             number_passes);
```

ufoProcessSetNumberRenderPass sets the number of separate rendering passes, and may be called within *ufoProcessDefine* to set the number of passes in all renders, or within *ufoProcessPreRender* to set the number of passes in the following render.

ufoProcessSetRenderPassMode

```
void ufoProcessSetRenderPassMode(
     ufoProcess      process_instance,
     int             pass_number,
     ufoPassMode     pass_mode);
```

Refer to the following table for command descriptions:

Command	Description
`pass_number`	pass for which the mode is being set
`ufoPassMode`	
`ufoHorizontalPass`	horizontal line scanning (default)
`ufoVerticalPass`	vertical line scanning

ufoProcessSetRenderPassMode sets the rendering of a particular pass to be either horizontal (the default) or vertical.

If the pass mode is set to *ufoVerticalPass* then the X and Y arguments passed to UFO render user functions are swapped for that pass.

ufoProcessSetRenderPassMode may be called within *ufoProcessDefine* to affect all renders, or within *ufoProcessPreRender* to affect the subsequent render.

ufoProcessSetTilingAllow

```
void ufoProcessSetTilingAllow(
    ufoProcess        process_instance,
    int               tiling_allow);
```

Refer to the following table for command descriptions:

Command	Description
`tiling_allow`	1: allow tiling/slicing (default) 0: do not allow

ufoProcessSetTilingAllow sets whether tiling or slicing is allowed when rendering.

By default, *TilingAllow* is on. If tiling is not allowed, then rendering of rectangular regions will not be segmented into slices or tiles. If allowed, slicing is done on processes which are slow to allow checking for a user interrupt, and tiling is optionally performed at the user's request to subdivide a high resolution render into a set of lower resolution renders.

ufoProcessSetTilingAllow may be called within *ufoProcessDefine*, to set for all renders, within *ufoProcessPreRender* to set before each frame, or within *ufoProcessPreRenderPass* to set before each render pass.

ufoProcessSetPixelTypeCombinations

```
void ufoProcessSetPixelTypeCombinations(
    ufoProcess        process_instance,
    int               number_combinations,
```

```
    ufoPixelType      *input_combinations,
    ufoPixelType      *output_combinations,
    ufoPixelType      *mask_combinations);
```

Refer to the following table for command descriptions:

Command	Description
`number_combinations`	number of combinations supported
`input_combinations`	2D array of pixel types [number_inputs] x [number_combinations]
`output_combinations`	2D array of pixel types [number_outputs] x [number_combinations]
`mask_combinations`	set to null

ufoProcessSetPixelTypeCombinations sets the combinations of input and output pixel types supported by the UFO process render functions. The *mask_combinations* argument should be set to null.

It is recommended that in place of using this function, you should supply the user function *ufoProcessSpecifyConvertPixelTypes* (see above). *ufoProcessSpecifyConvertPixelTypes* is easier to use and gives you more precise control of the pixel types of the rasters supplied to the UFO process.

By default, the process supports any pixel type on all inputs and outputs.

ufoProcessSetPixelTypeCombinations may be called within *ufoProcessDefine*.

ufoProcessRasterInDefine

```
void ufoProcessRasterInDefine(
    ufoProcess      process_instance,
    int             input_index,
    char            *id,
    char            *name,
    ufoCompComb     suggested_components);
```

Refer to the following table for command descriptions:

Command	Description
`input_index`	index (0..number_inputs-1) of raster input
`id`	unique (within process) id of input with no spaces
`name`	user interface title of input
`suggested_components`	suggested working pixel components

Command	Description
`ufoCompComb`	
`ufoAnyCompComb`	expects any pixel type
`ufoRGBCompComb`	expects RGB pixel type
`ufoRGBACompComb`	expects RGBA pixel type
`ufoACompComb`	expects A pixel type

ufoProcessRasterInDefine defines one of the (0..*number_inputs-1*) raster inputs as set in the *ufoProcessCreate* function call.

The *suggested_components* argument indicates which component combination of the raster the UFO process expects to work on. This does not force the pixel type of raster supplied when rendering; for this use *ufoProcesSetPixelTypeCombinations*.

ufoProcessRasterInDefine must be called within *ufoProcessDefine* for each raster input.

ufoProcessSetRasterInEdgeMode

```
void ufoProcessSetRasterInEdgeMode(
    ufoProcess        process_instance,
    int               input_index,
    ufoEdgeMode       edge_mode);
```

Refer to the following table for command descriptions:

Command	Description
`ufoEdgeMode`	
`ufoEdgeNoClamp`	Do not clamp output render to input bounds
`ufoEdgeNoClamp`	Clamp output render to input bounds (default)

If an effect can render to an output region which can be outside the bounds of this input, set the edge mode to *ufoEdgeNoClamp*.

By default *ufoProcessSetRasterInEdgeMode* is set to *ufoEdgeClamp*, which means that an output render region is automatically clamped to the limits of the input.

ufoProcessSetRasterInEdgeMode may be called within *ufoProcessDefine*.

ufoProcessSetRasterInOptional

```
void ufoProcessSetRasterInOptional(
    ufoProcess        process_instance,
    int               input_index,
    int               optional);
```

Refer to the following table for command descriptions:

Command	Description
`optional`	0: Not Optional (Default) 1: Optional

ufoProcessSetRasterInOptional sets the specified input to be optional or not. Optional inputs do not need to be connected in order for the UFO process effect to work.

If an input is set to optional, then *ufoProcessIsRasterInDefined* should be called to check for whether it is connected or not.

ufoProcessSetRasterInOptional may be called within *ufoProcessDefine*.

ufoProcessSetRasterInOptionalDefault

```
void ufoProcessSetRasterInOptional(
    ufoProcess     process_instance,
    int            input_index,
    int            option_select);
```

Refer to the following table for command descriptions:

Command	Description
`option_select`	0: Not Selected (Default) 1: Selected

ufoProcessSetRasterInOptional sets the default selection for the optional input (set by *ufoProcessSetRasterInOptionalDefault*).

ufoProcessSetRasterInOptionalDefault may be called within *ufoProcessDefine*.

ufoProcessIsRasterInDefined

```
int ufoProcessIsRasterInDefined(
    ufoProcess     process_instance,
    int            input_index);
```

ufoProcessIsRasterInDefined inquires if an optional raster input is connected and set.

These functions should be called for those inputs that have been set optional by *ufoProcessSetRasterInOptional* to see if the input has been connected and selected.

ufoProcessIsRasterInDefined may be called within *ufoProcessRenderPixel, ufoProcessRenderLine, ufoProcessRenderRectangle, ufoProcessCanAvoidRendering, ufoProcessPreRender, ufoProcessPostRender, ufoProcessPreRenderPass, ufoProcessPostRenderPass, ufoProcessCalcDefinedRectangle*, *ufoProcessCalcNeedRectangles*, *ufoProcessCalcDefinedPixelTypes,* and *ufoProcessSpecifyConvertPixelTypes*.

ufoProcessIsRasterInDirty

```
intufoProcessIsRasterInDirty(
    ufoProcess      process_instance,
    int             input_index);
```

ufoProcessIsRasterInDirty returns a value of 1 if input is dirty and a value of 0 if input is not dirty.

Inquire if raster input is dirty. An input is not dirty if it hasn't changed since the last render and its dirty status was reset with *ufoProcessResetRasterInDirty*. This may occur during consecutive renders at the same frame during interactive editing, or if there is a hold (or freeze) on the input in a sequence.

ufoProcessIsRasterInDirty may be called within *ufoProcessRenderPixel, ufoProcessRenderLine, ufoProcessRenderRectangle, ufoProcessCanAvoidRendering, ufoProcessPreRender, ufoProcessPostRender, ufoProcessPreRenderPass,ufoProcessPostRenderPass, ufoProcessCalcDefinedRectangle, ufoProcessCalcNeedRectangles, ufoProcessCalcDefinedPixelTypes,* and *ufoProcessSpecifyConvertPixelTypes*.

ufoProcessResetRasterInDirty

```
void ufoProcessIsRasterInDirty(
    ufoProcess        process_instance,
    int               input_index);
```

ufoProcessResetRasterInDirty resets dirty status of input, and may be called within *ufoProcessRenderPixel, ufoProcessRenderLine, ufoProcessRenderRectangle, ufoProcessCanAvoidRendering, ufoProcessPreRender, ufoProcessPostRender, ufoProcessPreRenderPass, ufoProcessPostRenderPass, ufoProcessCalcDefinedRectangle, ufoProcessCalcNeedRectangles, ufoProcessCalcDefinedPixelTypes,* and *ufoProcessSpecifyConvertPixelTypes*.

ufoProcessRasterOutDefine

```
void ufoProcessRasterOutDefine(
    ufoProcess        process_instance,
```

```
    int                 output_index,
    char                *id,
    char                *name,
    ufoCompComb         suggested_components);
```

Refer to the following table for command descriptions:

Command	Description
`output_index`	index (0..number_outputs-1) of raster output
`id`	unique (within process) id of output with no spaces
`name`	user interface title of output
`suggested_components`	suggested working pixel components
`ufoCompComb`	
`ufoAnyCompComb`	expects any pixel type
`ufoRGBCompComb`	expects RGB pixel type
`ufoRGBACompComb`	expects RGBA pixel type
`ufoACompComb`	expects A (alpha) pixel type

ufoProcessRasterOutDefine define one of the (0..*number_outputs-1*) raster outputs as set in the *ufoProcessCreate* function call.

The *suggested_components* argument indicates which component combination of the raster the UFO process expects to work on. This does not force the pixel type of raster supplied when rendering; for this use *ufoProcesSetPixelTypeCombinations*.

ufoProcessRasterOutDefine must be called within *ufoProcessDefine* for each raster output.

ufoProcessSetRasterOutEdgeMode

```
void ufoProcessSetRasterOutEdgeMode(
    ufoProcess      process_instance,
    int             input_index,
    ufoEdgeMode     edge_mode);
```

Refer to the following table for command descriptions:

Command	Description
`ufoEdgeMode`	
`ufoEdgeNoClamp`	Do not clamp output render to input bounds
`ufoEdgeNoClamp`	Clamp output render to input bounds (default)

If an effect can render to an output region which can be outside the bounds of this input, set the edge mode to *ufoEdgeNoClamp*.

By default *ufoProcessSetRasterInEdgeMode* is set to *ufoEdgeClamp*, which means that an output render region is automatically clamped to the limits of the input.

ufoProcessSetRasterInEdgeMode may be called within *ufoProcessDefine*.

ufoProcessGetRasterIn
ufoProcessGetRasterOut
ufoProcessGetObeyMatte

```
ufoRaster ufoProcessGetRasterIn(
    ufoProcess      process_instance,
    int             input_index);
ufoRaster ufoProcessGetRasterOut(
    ufoProcess      process_instance,
    int             output_index);
ufoRaster ufoProcessGetObeyMatte(
    ufoProcess      process_instance);
```

ufoProcessGetRasterIn, ufoProcessGetRasterOut and *ufoProcessGetObeyMatte* return a handle to a ufoRaster instance. Use them to return the raster handle for input or output during rendering.

ufoProcessGetObeyMatte should be used where a process handles its own matte protection. See also *ufoProcessHandleOwnProtection* on page 739, *ufoProcessIsObeyMatteDefined* on page 739, and *ufoProcessIsObeyMatteReversed* on page 740.

The raster handle can be passed in calls to raster functions that take a *ufoRaster* handle as their first argument. These are functions to set and get pixel values, to inquire raster attributes and buffer pointers, and for raster copying.

ufoProcessGetRasterIn and *ufoProcessGetRasterOut* may be called within *ufoProcessRenderPixel, ufoProcessRenderLine, ufoProcessRenderRectangle,*

ufoProcessPreRender, ufoProcessPostRender, ufoProcessPreRenderPass, ufoProcessPostRenderPass or from within any editor user function or event function in the editor GUI.

ufoProcessGetRasterInAtFrame ufoProcessReleaseRasterInAtFrame

```
ufoRaster ufoProcessGetRasterIn(
ufoProcess process_instance,
int input_index,
int frame,
int field);
ufoProcessReleaseRasterIn(
ufoProcess process_instance,
int input_index,
int frame,
int field);
```

ufoProcessGetRasterInAtFrame returns a handle to a *ufoRaster* instance for a specified frame and field.

The field argument is only relevant during field rendering in which case it should be set to 1 or 2. *ufoProcessGetRasterInAtFrame* returns the raster for an input at the specified frame (and field). The function includes a field argument which is used when field rendering (see *ufoProcessFieldRendering*).

Each call to *ufoProcessGetRasterInAtFrame* with a unique frame (and field) will lock a raster in memory by the application memory manager until *ufoProcessReleaseRasterInAtFrame* is subsequently called with that frame (and field). It is very important to use this function to unlock memory from the memory manager and not keep many frames locked in memory simultaneously. If you don't call *ufoProcessReleaseRasterInAtFrame* then all accessed frame rasters will remain locked until after the UFO process has finished rendering at that frame.

Temporal access only works if the input is connected directly to a clip. *ufoProcessIsRasterInTemporal* can be used to test this.

The raster handle returned by can be passed in calls to raster functions that take a *ufoRaster* handle as their first argument. These are functions to set and get pixel values, to inquire raster attributes and buffer pointers, and for raster copying.

ufoProcessGetRasterInAtFrame and *ufoProcessReleaseRasterInAtFrame* may be called within *ufoProcessRenderPixel*, *ufoProcessRenderLine*, *ufoProcessRenderRectangle*, *ufoProcessPreRender*, *ufoProcessPostRender*, *ufoProcessPreRenderPass*, and *ufoProcessPostRenderPass*.

ufoProcessIsRasterInTemporal

```
int ufoProcessIsRasterInTemporal(
    ufoProcess      process_instance,
    int             input_index);
```

Returns whether a UFO process raster input can be temporally accessed. For more information, see *ufoProcessGetRasterInAtFrame ufoProcessReleaseRasterInAtFrame* on page 738.

ufoProcessIsRasterInTemporal may be called within *ufoProcessRenderPixel*, *ufoProcessRenderLine*, *ufoProcessRenderRectangle*, *ufoProcessPreRender*, *ufoProcessPostRender*, *ufoProcessPreRenderPass*, *ufoProcessPostRenderPass*, *ufoProcessCanAvoidRendering* or from within any editor user function or event function in the editor GUI.

ufoProcessGetRasterInTemporalLimits

```
void ufoProcessGetRasterInTemporalLimits(
    ufoProcess      process_instance,
    int             input_index,
    int *           first_frame,
    int *           last_frame);
```

Returns the frame limits available for temporal access on an inputs.

ufoProcessGetRasterInTemporalLimits may be called within *ufoProcessRenderPixel*, *ufoProcessRenderLine*, *ufoProcessRenderRectangle*, *ufoProcessPreRender*, *ufoProcessPostRender*, *ufoProcessPreRenderPass*, *ufoProcessPostRenderPass*, *ufoProcessCanAvoidRendering*, or from within any editor user function or event function in the editor GUI.

ufoProcessHandleOwnProtection

```
void ufoProcessHandleOwnProtection(
    ufoProcess      process_instance);
```

Sets the UFO process to handle its own obey matte protection, bypassing Media Illusion's internal obey matte functionality.

ufoProcessHandleOwnProtection may be called within *ufoProcessDefine*.

ufoProcessIsObeyMatteDefined

```
int ufoProcessIsObeyMatteDefined(
    ufoProcess        process_instance);
```

ufoProcessIsObeyMatteDefined inquires if an optional obey matte raster is connected and set, and may be called within *ufoProcessRenderPixel*, *ufoProcessRenderLine*, *ufoProcessRenderRectangle*, *ufoProcessCanAvoidRendering*, *ufoProcessPreRender*, *ufoProcessPostRender*,

ufoProcessPreRenderPass, ufoProcessPostRenderPass, ufoProcessCalcDefinedRectangle, ufoProcessCalcNeedRectangles, ufoProcessCalcDefinedPixelTypes, and *ufoProcessSpecifyConvertPixelTypes*.

ufoProcessIsObeyMatteReversed

```
int ufoProcessIsObeyMatteReversed(
    ufoProcess        process_instance);
```

ufoProcessIsObeyMatteReversed inquires if an optional obey matte is reversed, and may be called within *ufoProcessRenderPixel, ufoProcessRenderLine, ufoProcessRenderRectangle, ufoProcessCanAvoidRendering, ufoProcessPreRender, ufoProcessPostRender, ufoProcessPreRenderPass, ufoProcessPostRenderPass, ufoProcessCalcDefinedRectangle, ufoProcessCalcNeedRectangles*, *ufoProcessCalcDefinedPixelTypes,* and *ufoProcessSpecifyConvertPixelTypes*.

ufoProcessIsObeyMatteDirty

```
int ufoProcessIsObeyMatteDirty(
    ufoProcess        process_instance);
```

ufoProcessIsObeyMatteDirty inquires if the obey matte raster input is dirty.

An input is not dirty if it has not changed since the last render and its dirty status was reset with *ufoProcessResetObeyMatteDirty*.

A value of 1 is returned if the obey matte raster is dirty or 0 if it is not dirty.

May be called within *ufoProcessRenderPixel, ufoProcessRenderLine, ufoProcessRenderRectangle, ufoProcessCanAvoidRendering, ufoProcessPreRender, ufoProcessPostRender, ufoProcessPreRenderPass, ufoProcessPostRenderPass, ufoProcessCalcDefinedRectangle,* and *ufoProcessCalcNeedRectangles*.

ufoProcessResetObeyMatteDirty

```
void ufoProcessResetObeyMatteDirty(
    ufoProcess        process_instance);
```

ufoProcessResetObeyMatteDirty resets the dirty status of obey matte raster, and may be called within *ufoProcessRenderPixel, ufoProcessRenderLine, ufoProcessRenderRectangle, ufoProcessCanAvoidRendering, ufoProcessPreRender, ufoProcessPostRender, ufoProcessPreRenderPass, ufoProcessPostRenderPass, ufoProcessCalcDefinedRectangle, ufoProcessCalcNeedRectangles*, *ufoProcessCalcDefinedPixelTypes,* and *ufoProcessSpecifyConvertPixelTypes*.

ufoProcessParamDefine

```
void ufoProcessParamDefine(
    ufoProcess        process_instance,
```

```
    int                 param_index,
    int                 group_index,
    char                *id,
    char                *name,
    ufoParamType        type);
```

Refer to the following table for command descriptions:

Command	Description
`param_index`	index (0..number_parameters-1) of parameter
`group_index`	index (0..number_parameter_groups) of parameter's group
`id`	unique (within process) id of parameter with no spaces
`name`	user interface title of parameter
`type`	parameter type
`ufoParamType`	
`ufoBooleanParam`	true/false on/off
`ufoIntegerParam`	integer
`ufoFloatParam`	floating point
`ufoNormalisedParam`	floating point normalized (0..1)
`ufoAngleParam`	floating point angle
`ufoColourRGBParam`	RGB colour
`ufoColourCMYParam`	CMY colour
`ufoColourHSVParam`	HSV colour
`ufoColourHLSParam`	HLS colour
`ufoRectangleParam`	floating point rectangle
`ufoPos3dParam`	floating point 3D position
`ufoScale3dParam`	floating point 3D scale
`ufoPos2dParam`	floating point 2D position
`ufoScale2dParam`	floating point 2D scale
`ufoStringParam`	character string

ufoProcessParamDefine defines one of (0..*number_parameters*) parameters set in *ufoProcessCreate*.

ufoProcessParamDefine must be called within *ufoProcessDefine* for each parameter, except for parameters which are of the type *ufoEnumParam*, which has a special define function, *ufoProcessEnumParamDefine*.

ufoProcessEnumParamDefine

```
void ufoProcessEnumParamDefine(
    ufoProcess        process_instance,
    int               param_index,
    int               group_index,
    char              *id,
    char              *name,
    int               number_values,
    char**            enum_value_text);
```

Refer to the following table for command descriptions:

Command	Description
`param_index`	index (0..number_parameters-1) of parameter
`group_index`	index (0..number_parameter_groups) of parameter's group
`id`	unique (within process) id of parameter with no spaces
`name`	user interface title of parameter
`number_values`	number of values in the enumeration
`num_value_text`	array of (null terminated) strings representing enumeration values

ufoProcessEnumParamDefine defines an enumerated parameter, one of (0..*number_parameters*) parameters set in *ufoProcessCreate*.

ufoProcessEnumParamDefine must be called within *ufoProcessDefine* for each *ufoEnumParam* type parameter.

ufoProcessParamDefine must be called within *ufoProcessDefine* for each parameter, except for parameters which are of the type *ufoEnumParam*, which has a special define function, *ufoProcessEnumParamDefine*.

ufoProcessParamGroupDefine

```
void ufoProcessParamGroupDefine(
    ufoProcess        process_instance,
```

```
    int                 group_index,
    char                *id,
    char                *name);
```

Refer to the following table for command descriptions:

Command	Description
`group_index`	index (0..number_parameter_groups) of parameter group
`id`	unique (within process) ID of parameter group with no spaces
`name`	user interface title of parameter group

ufoProcessParamGropDefine defines a parameter group, one of (0..*number_parameter_groups*) parameter groups set in *ufoProcessCreate*.

ufoProcessParamGroupDefine must be called within *ufoProcessDefine* for each parameter group.

ufoProcessSetParamAnimAllow

```
void ufoProcessSetParamAnimAllow(
    ufoProcess          process_instance,
    int                 param_index,
    int                 anim_allow);
```

Refer to the following table for command descriptions:

Command	Description
`param_index`	index (0..number_parameters-1) of parameter
`anim_allow`	1: allow animation (default), 0: do not

ufoProcessSetParamAnimAllow allows or disallows parameter animation, and may be called within *ufoProcessDefine* after *ufoProcessParamDefine*.

ufoProcessSetParamAutoTransition

```
void ufoProcessSetParamAutoTransition(
    ufoProcess          process_instance,
    int                 param_index,
    int                 auto_transition);
```

Refer to the following table for command descriptions:

Command	Description
`param_index`	index (0..number_parameters-1) of parameter
`auto_transition`	1: auto transition on, 0: off (default)

ufoProcessSetParamAutoTransition automatically fits default parameter animation to a transition period when it is first used. Thereafter, the animation will have to be manually adjusted. This only works if the parameter has at least two keys set in its default values.

ufoProcessSetParamAutoTransition may be called within *ufoProcessDefine* after *ufoProcessParamDefine*.

ufoProcessSetParamLimits

```
void ufoProcessSetParamLimits(
    ufoProcess          process_instance,
    int                 param_index,
    ufoChannelIndex     channel_index,
    double              minimum,
    double              maximum,
    ufoLimits           Clamp clamp_flags);
```

Refer to the following table for command descriptions:

Command	Description
`param_index`	index (0..number_parameters-1) of parameter
`channel_index`	index of channel to which limits will be applied (usually 0)
`minimum`	lower limit
`maximum`	upper limit
`clamp_flags`	sets clamping on lower or upper limits
`ufoChannelIndex`	see *ufoTypes.h* for complete set of definitions for channel indices
`ufoLimitsClamp`	
`ufoLimitsClampNone`	no clamping
`ufoLimitsClampMin`	clamp lower limit only

Command	Description
`ufoLimitsClampMax`	clamp upper limit only
`ufoLimitsClampBoth`	clamp both limits

ufoProcessSetParamLimits sets limits and clamping for a parameter. This function should be called for parameters of types *ufoIntegerParam, ufoFloatParam, ufoAngleParam, ufoRectangleParam, ufoPos3dParam, ufoScale3dParam, ufoPos2dParam*, and *ufoScale2dParam*, where limits must be defined for user interface purposes. *clamp_flags* can be set to optionally clamp or not clamp values when they are inquired.

ufoProcessSetParamLimits maybe called within *ufoProcessDefine* after *ufoProcessParamDefine.*

ufoProcessSetParamColumnHint

```
void ufoProcessSetParamColumnHint(
    ufoProcess        process_instance,
    int               param_index,
    int               column);
```

Refer to the following table for command descriptions:

Command	Description
`param_index`	index (0..number_parameters-1) of parameter
`column`	preferred column

ufoProcessSetParamAnimAllow will hint to the host application the column in which the standard editor for the parameter should be built. This hint may be ignored.

ufoProcessSetParamAnimAllow may be called within *ufoProcessDefine* after *ufoProcessParamDefine.*

ufoProcessSetParamGang
ufoProcessSetParamGangLabel

```
void ufoProcessSetParamGang(
    ufoProcess        process_instance,
    int               param_index,
    int               number_params);
```

Refer to the following table for command descriptions:

Command	Description
`param_index`	index (0..number_parameters-1) of parameter
`number_params`	number of parameters to gang

```
void ufoProcessSetParamGangLabel(
    ufoProcess        process_instance,
    int               param_index,
    char *            label);
```

ufoProcessSetParamGang will gang certain types of parameters together. The number of ganged parameters is defined by *number_params* and start from *param_index*. The supported types are: *ufoIntegerParam*, *ufoFloatParam*, *ufoNormalisedParam*, and *ufoAngleParam*. Ganged parameters have an extra control to allow all the sliders to be moved simultaneously, such that when the gang control is switched on and any one ganged parameter is changed, then all ganged parameters are changed by the same amount.

ufoProcessSetParamGangLabel sets a collective title for the ganged parameters starting at param_index.

ufoProcessSetParamGang and *ufoProcessSetParamGangLabel* may be called within *ufoProcessDefine* after *ufoProcessParamDefine*.

ufoProcessHideParam
ufoProcessShowParam

```
void ufoProcessHideParam(
    ufoProcess        process_instance,
    int               param_index);
void ufoProcessShowParam(
    ufoProcess        process_instance,
    int               param_index);
```

Refer to the following table for command descriptions:

Command	Description
`param_index`	index (0..number_parameters-1) of parameter

ufoProcessSetHideParam and *ufoProcessShowParam* can be used to hide or show parameters within the application interface, and may be called within *ufoProcessDefine* after *ufoProcessParamDefine*, or from within any editor user function or event function in the editor GUI.

ufoProcessSetParamDefaultValue
ufoProcessSetParamDefaultValues
ufoProcessSetStringParamDefaultValue

```
void ufoProcessSetParamDefaultValue(
        ufoProcess          process_instance,
        int                 param_index,
        ufoChannelIndex     channel_index,
        double              value);
```

Refer to the following table for command descriptions:

Command	Description
`param_index`	index (0..number_parameters-1) of parameter
`channel_index`	index of channel to which default value will be applied usually 0)
`value`	default value
`ufoChannelIndex`	see *ufoTypes.h* for complete set of definitions for channel indices

```
void ufoProcessSetParamDefaultValues(
    ufoProcess        process_instance,
    int               param_index,
    double            values[]);
```

Refer to the following table for command descriptions:

Command	Description
`param_index`	index (0..number_parameters-1) of parameter
`values`	default values for all parameter channels

```
void ufoProcessSetStringParamDefaultValue(
    ufoProcess        process_instance,
    int               param_index,
    const char *      string);
```

Refer to the following table for command descriptions:

Command	Description
`param_index`	index (0..number_parameters-1) of parameter
`string`	default string value

ufoProcessSetParamDefaultValue(s) and *ufoProcessSetStringParamDefaultValue* set multiple default values for parameters.

ufoProcessSetParamDefaultValue(s) and *ufoProcessSetStringParamDefaultValue* should be called within *ufoProcessDefine* after *ufoProcessParamDefine*.

ufoProcessSetParamDefaultKey
ufoProcessSetParamDefaultKeys
ufoProcessSetStringParamDefaultKey

```
void ufoProcessSetParamDefaultValue(
    ufoProcess          process_instance,
    int                 param_index,
    ufoChannelIndex     channel_index,
    double              value,
    double              time);
```

Refer to the following table for command descriptions:

Command	Description
`param_index`	index (0..number_parameters-1) of parameter
`channel_index`	index of channel to which default key value will be applied (usually 0)
`value`	default key value
`time`	key time
`ufoChannelIndex`	see *ufoTypes.h* for complete set of definitions for channel indices

```
void ufoProcessSetParamDefaultValues(
    ufoProcess          process_instance,
    int                 param_index,
    double              values[],
    double              time);
```

Refer to the following table for command descriptions:

Command	Description
`param_index`	index (0..number_parameters-1) of parameter
`values`	key values for all parameter channels
`time`	key time

```
void ufoProcessSetStringParamDefaultKey(
    ufoProcess          process_instance,
    int                 param_index,
    const char *        string,
    double              time);
```

Refer to the following table for command descriptions:

Command	Description
`param_index`	index (0..number_parameters-1) of parameter
`string`	default string key value
`time`	key time

ufoProcessSetParamDefaultKey(s) and *ufoProcessSetStringParamDefaultKey* set multiple default key values for parameters which have more than one channel.

ufoProcessSetParamDefaultKey(s) and *ufoProcessSetStringParamDefaultKey* should be called within *ufoProcessDefine* after *ufoProcessParamDefine*.

ufoProcessGetParamValue
ufoProcessGetParamValues
ufoProcessGetStringParamValue

```
double ufoProcessGetParamValue(
    ufoProcess          process_instance,
    int                 param_index,
    ufoChannelIndex     channel_index);
```

Refer to the following table for command descriptions:

Command	Description
`param_index`	index (0..number_parameters-1) of parameter
`channel_index`	index of channel from which current value will be returned (usually 0)
`ufoChannelIndex`	see *ufoTypes.h* for complete set of definitions for channel indices

```
void ufoProcessGetParamValues(
    ufoProcess          process_instance,
    int                 param_index,
    double              values[]);
```

Refer to the following table for command descriptions:

Command	Description
`param_index`	index (0..number_parameters-1) of parameter
`values`	returned current values for all parameter channels

```
char * ufoProcessGetStringParamValue(
    ufoProcess          process_instance,
    int                 param_index);
```

Refer to the following table for command descriptions:

Command	Description
`param_index`	index (0..number_parameters-1) of parameter

Get parameter values(s) at the current time. Use *ufoProcessGetParamValues* for getting multiple values for parameters which have more than one channel. Use *ufoProcessGetStringParamValue* to get the current value of a *ufoStringParam* parameter, and returns a character string allocated with malloc which the UFO has the responsibility to free.

ufoProcessGetParamValue(s) and *ufoProcessGetStringParamValue* may be called within *ufoProcessRenderPixel*, *ufoProcessRenderLine*, *ufoProcessRenderRectangle*, *ufoProcessCanAvoidRendering*, *ufoProcessPreRender*, *ufoProcessPostRender*, *ufoProcessPreRenderPass*, *ufoProcessPostRenderPass*, *ufoProcessCalcDefinedRectangle*,

ufoProcessCalcNeedRectangles, *ufoProcessCalcDefinedPixelTypes*, *ufoProcessSpecifyConvertPixelTypes*, and from within any editor user function or event function in the editor GUI.

ufoProcessGetParamValueAtTime
ufoProcessGetParamValuesAtTime
ufoProcessGetStringParamValueAtTime

```
double ufoProcessGetParamValueAtTime(
    ufoProcess          process_instance,
    int                 param_index,
    ufoChannelIndex     channel_index,
    double              time);
```

Refer to the following table for command descriptions:

Command	Description
`param_index`	index (0..number_parameters-1) of parameter
`channel_index`	index of channel from which current value will be returned (usually 0)
`time`	time at which to evaluate value
`ufoChannelIndex`	see *ufoTypes.h* for complete set of definitions for channel indices

```
void ufoProcessGetParamValuesAtTime(
    ufoProcess          process_instance,
    int                 param_index,
    double              values[],
    double              time);
```

Refer to the following table for command descriptions:

Command	Description
`param_index`	index (0..number_parameters-1) of parameter
`values`	returned current values for all parameter channels
`time`	time at which to interpolate values

```
char * ufoProcessGetStringParamValueAtTime(
    ufoProcess          process_instance,
    int                 param_index,
```

```
    double              time);
```

Refer to the following table for command descriptions:

Command	Description
`param_index`	index (0..number_parameters-1) of parameter
`time`	time at which to evaluate value

ufoProcessGetParamValue(s)AtTime gets parameter values(s) at a specified time. Use *ufoProcessGetParamValuesAtTime* for getting multiple values for parameters which have more than one channel. Use *ufoProcessGetStringParamValueAtTime* to get the value of a *ufoStringParam* parameter at a specified time, and returns a character string allocated with malloc which the UFO has the responsibility to free.

ufoProcessGetParamValue(s)AtTime may be called within *ufoProcessRenderPixel*, *ufoProcessRenderLine*, *ufoProcessRenderRectangle*, *ufoProcessCanAvoidRendering*, *ufoProcessPreRender*, *ufoProcessPostRender*, *ufoProcessPreRenderPass*, *ufoProcessPostRenderPass*, *ufoProcessCalcDefinedRectangle*, *ufoProcessCalcNeedRectangles*, *ufoProcessCalcDefinedPixelTypes*, *ufoProcessSpecifyConvertPixelTypes*, and from within any editor user function or event function in the editor GUI.

ufoProcessSetParamValue
ufoProcessSetParamValues
ufoProcessSetStringParamValue

```
void ufoProcessSetParamValue(
    ufoProcess          process_instance,
    int                 param_index,
    ufoChannelIndex     channel_index,
    double              value);
```

Refer to the following table for command descriptions:

Command	Description
`param_index`	index (0...number_parameters-1) of parameter

```
void ufoProcessSetParamValues(
    ufoProcess          process_instance,
    int                 param_index,
    double              value[]);
```

Refer to the following table for command descriptions:

Command	Description
`param_index`	index (0...number_parameters-1) of parameter

```
void ufoProcessSetStringParamValue(
    ufoProcess          process_instance,
    int                 param_index,
    const char *        string);
```

Refer to the following table for command descriptions:

Command	Description
`param_index`	index (0...number_parameters-1) of parameter

ufoProcessSetParamValue(s) and *ufoProcessSetStringParamValue* set animation key values for a parameter at the current time.

ufoProcessSetParamValue(s) and *ufoProcessSetStringParamValue* may be called within *ufoEditorMouseDown()*, *ufoEditorMouseUp()*, *ufoEditorMouseDrag()*, *ufoEditorMousePosition()*, *ufoEditorKeyEvent()*, or from within any event function in the UFO editor GUI.

ufoProcessSetParamKey
ufoProcessSetParamKeys
ufoProcessSetStringParamKey

```
void ufoProcessSetParamKey(
    ufoProcess          process_instance,
    int                 param_index,
    ufoChannelIndex     channel_index,
    double              value,
    double              time);
```

Refer to the following table for command descriptions:

Command	Description
`param_index`	index (0...number_parameters-1) of parameter
`channel_index`	index of channel to which key value will be applied (usually 0)

Command	Description
`value`	key value
`time`	key time

```
void ufoProcessSetParamKeys(
    ufoProcess        process_instance,
    int               param_index,
    double            values(),
    double            time);
```

Refer to the following table for command descriptions:

Command	Description
`param_index`	index (0...number_parameters-1) of parameter
`values`	key values for all parameter channels
`time`	key time

```
void ufoProcessSetStringParamKey(
    ufoProcess        process_instance,
    int               param_index,
    const char *      value,
    double            time);
```

Refer to the following table for command descriptions:

Command	Description
`param_index`	index (0...number_parameters-1) of parameter
`string`	string key value
`time`	key time

ufoProcessSetParamKey(s) and *ufoProcessSetStringParamKey* set animation key values for a parameter.

ufoProcessSetParamKeys may be called within *ufoEditorMouseDown()*, *ufoEditorMouseUp()*, *ufoEditorMouseDrag()*, *ufoEditorMousePosition()*, *ufoEditorKeyEvent()*, or from within any event function in the UFO editor GUI.

ufoProcessIsParamSet

```
int ufoProcessIsParamSet(
    ufoProcess          process_instance,
    int                 param_index);
```

Refer to the following table for command descriptions:

Command	Description
param_index	index (0..number_parameters-1) of parameter

ufoProcesslsParamSet returns whether a parameter has been set since default initialization.

ufoProcesslsParamSetmay be called within *ufoProcessRenderPixel*, *ufoProcessRenderLine*, *ufoProcessRenderRectangle*, *ufoProcessCanAvoidRendering*, *ufoProcessPreRender*, *ufoProcessPostRender*, *ufoProcessPreRenderPass*, *ufoProcessPostRenderPass*, *ufoProcessCalcDefinedRectangle*, *ufoProcessCalcNeedRectangles*, *ufoProcessCalcDefinedPixelTypes*, *ufoProcessSpecifyConvertPixelTypes*, and from within any editor user function or event function in the editor GUI.

ufoProcessGetParamMinMaxKeyTime
ufoProcessGetChanMinMaxKeyTime

```
void ufoProcessGetParamMinMaxKeyTime(
    ufoProcess          process_instance,
    int                 param_index,
    double *            min_time,
    double *            max_time);
void ufoProcessGetChanMinMaxKeyTime(
    ufoProcess          process_instance,
    int                 param_index,
    ufoChannelIndex     channel_index,
    double *            min_time,
    double *            max_time);
```

Refer to the following table for command descriptions:

Command	Description
param_index	index (0..number_parameters-1) of parameter
channel_index	index of channel

ufoProcessGetParamMinMaxKeyTime and *ufoProcessGetChanMinMaxKeyTime* return the frame times of the minimum and maximum keys for the parameter or channel in time, respectively.

May be called within *ufoProcessRenderPixel*, *ufoProcessRenderLine*, *ufoProcessRenderRectangle*, *ufoProcessCanAvoidRendering*, *ufoProcessPreRender*, *ufoProcessPostRender*, *ufoProcessPreRenderPass*, *ufoProcessPostRenderPass*, *ufoProcessCalcDefinedRectangle*, *ufoProcessCalcNeedRectangles*, *ufoProcessCalcDefinedPixelTypes*, *ufoProcessSpecifyConvertPixelTypes*, and from within any editor user function or event function in the editor GUI.

ufoProcessSetUserData

```
void ufoProcessSetUserData(
    ufoProcess        process_instance,
    void              *user_data);
```

ufoProcessSetUserData sets user data to be associated with this instance of the process.

User data may be set to null.

If user data is set and is required to be copied when copies are made of the process, then you should supply *ufoProcessCopyUserData*. If user data should be tidied when the process is put away, you should supply the *ufoProcessDeleteUserData* user function.

ufoProcessSetUserData may be called within any user function.

ufoProcessGetUserData

```
void* ufoProcessGetUserData(
    ufoProcess        process_instance);
```

ufoProcessGetUserData returns user data previously set with *ufoProcessSetUserData* for this process instance. Returns null if no user data is currently set.

ufoProcessGetRenderMode

```
ufoRenderMode ufoProcessGetRenderMode(
    ufoProcess        process_instance);
```

Refer to the following table for command descriptions:

Command	Description
`ufoRenderMode`	
`ufoPreviewFrame`	single frame interactive-editing preview

Command	Description
`ufoPreviewSequence`	sequence preview
`ufoFullFrame`	single frame full render, no render scaling
`ufoFullSequence`	sequence full render, no render scaling

ufoProcessGetRenderMode returns the current render mode. This can be either preview and full, or single and sequence.

ufoProcessGetRenderMode is useful for ignoring parameters in a full render which control accelerated methods used in editing or previewing.

ufoProcessGetRenderMode may be called within *ufoProcessRenderPixel*, *ufoProcessRenderLine*, *ufoProcessRenderRectangle*, *ufoProcessCanAvoidRendering*, *ufoProcessPreRender*, *ufoProcessPostRender*, *ufoProcessPreRenderPass*, *ufoProcessPostRenderPass*, *ufoProcessCalcDefinedRectangle*, *ufoProcessCalcNeedRectangles*, *ufoProcessCalcDefinedPixelTypes*, *ufoProcessSpecifyConvertPixelTypes*, and from within any editor user function or event function in the editor GUI.

ufoProcessGetRenderScale

```
void ufoProcessGetRenderScale(
    ufoProcess        process_instance,
    double            *x_scale,
    double            *y_scale);
```

Refer to the following table for command descriptions:

Command	Description
`x_scale`	x scale of current render (x_scale <= 1)
`y_scale`	y scale of current render (y_scale <= 1)

ufoProcessGetRenderScale returns the X and Y scale of the current render and may be called within *ufoProcessRenderPixel*, *ufoProcessRenderLine*, *ufoProcessRenderRectangle*, *ufoProcessCanAvoidRendering*, *ufoProcessPreRender*, *ufoProcessPostRender*, *ufoProcessPreRenderPass*, *ufoProcessPostRenderPass*, *ufoProcessCalcDefinedRectangle*, *ufoProcessCalcNeedRectangles*, *ufoProcessCalcDefinedPixelTypes*, *ufoProcessSpecifyConvertPixelTypes*, and from within any editor user function or event function in the editor GUI.

ufoProcessGetAspectRatioCorrection

```
double ufoProcessGetAspectRatioCorrection(
    ufoProcess        process_instance);
```

ufoProcessGetAspectRatioCorrection returns the aspect ratio correction factor to be applied during rendering.

The value to be returned is the scale to be applied to the X pixel dimension in order to display it with the correct aspect. For example, if a PAL 720x576 image is to be displayed at 16:9, then the aspect ratio correction returned will be (576/720)*(16/9) = 1.42222.

Any effects involving transformations, such as rotation will need to take this aspect ratio into account in order to appear correctly when displayed in the final aspect ratio.

ufoProcessGetAspectRatioCorrection may be called within *ufoProcessRenderPixel*, *ufoProcessRenderLine*, *ufoProcessRenderRectangle*, *ufoProcessCanAvoidRendering*, *ufoProcessPreRender*, *ufoProcessPostRender*, *ufoProcessPreRenderPass*, *ufoProcessPostRenderPass*, *ufoProcessCalcDefinedRectangle*, *ufoProcessCalcNeedRectangles*, *ufoProcessCalcDefinedPixelTypes*, *ufoProcessSpecifyConvertPixelTypes*, and from within any editor user function or event function in the editor GUI.

ufoProcessGetRenderPass

```
int ufoProcessGetRenderPass(
    ufoProcess        process_instance);
```

ufoProcessGetRenderPass returns the current render pass, and may be called within *ufoProcessRenderPixel, ufoProcessRenderLine, ufoProcessRenderRectangle, ufoProcessPreRenderPass*, and *ufoProcessPostRenderPass*.

ufoProcessGetMinEffectLimit
ufoProcessGetMaxEffectLimit

```
double ufoProcessGetMinEffectLimit(
    ufoProcess        process_instance);
double ufoProcessGetMaxEffectLimit(
    ufoProcess        process_instance);
```

ufoProcessGetMinEffectLimit and *ufoProcessGetMaxEffectLimit* return the current render effect start and end frame limits.

These return a double - but values will very likely always be integer.

ufoProcessGetMinEffectLimit and *ufoProcessGetMaxEffectLimit* may be called within *ufoProcessRenderPixel*, *ufoProcessRenderLine*, *ufoProcessRenderRectangle*,*ufoProcessCanAvoidRendering*, *ufoProcessPreRenderPass*, *ufoProcessPostRenderPass*, *ufoProcessCalcDefinedRectangle*, *ufoProcessCalcNeedRectangles*, *ufoProcessCalcDefinedPixelTypes*, and *ufoProcessSpecifyConvertPixelType*.

ufoProcessCheckAbort

```
int ufoProcessCheckAbort(
    ufoProcess        process_instance);
    int               percentage_render_done);
```

Refer to the following table for command descriptions:

Command	Description
`percentage_render_done`	amount of frame rendered
`ufoProcessCheckAbort`	returns 1: continue rendering, and 0: stop rendering

ufoProcessCheckAbort checks for the user interrupting processing. An estimate of the percentage of the render done is passed for display purposes.

This function should only be used within *ufoProcessRenderRectangle* and the UFO should be defined with the default tiling method turned off. For more information, see *ufoProcessSetTilingAllow* on page 731.

If the UFO is using multi-threading, then this should only be called from the parent thread with which *ufoProcessRenderRectangle* was called.

ufoProcessCheckAbort may be called within *ufoProcessRenderRectangle*.

ufoProcessGetTime

```
double ufoProcessGetTime(
    ufoProcess        process_instance);
```

ufoProcessGetTime returns the current frame time.

The value returned is a double, but is probably always an integer or integer+0.5 aligned (in the case of field rendering).

ufoProcessGetTime may be called within *ufoProcessRenderPixel*, *ufoProcessRenderLine*, *ufoProcessRenderRectangle*, *ufoProcessCanAvoidRendering*, *ufoProcessPreRender*, *ufoProcessPostRender*, *ufoProcessPreRenderPass*, *ufoProcessPostRenderPass*, *ufoProcessCalcDefinedRectangle*, *ufoProcessCalcNeedRectangles*, *ufoProcessCalcDefinedPixelTypes*, *ufoProcessSpecifyConvertPixelTypes*, and from within any editor user function or event function in the editor GUI.

ufoProcessFieldRendering

```
int ufoProcessFieldRendering(
    ufoProcess        process_instance);
```

ufoProcessFieldRendering returns whether the current render is in field or frame mode.If in field mode The full output should still be rendered. This

function is mainly useful if you are using temporal frame access on your inputs in which case you may need to access inputs at field intervals.

ufoProcessFieldRendering may be called within *ufoProcessRenderPixel*, *ufoProcessRenderLine*, *ufoProcessRenderRectangle*, *ufoProcessCanAvoidRendering*, *ufoProcessPreRender*, *ufoProcessPostRender*, *ufoProcessPreRenderPass*, *ufoProcessPostRenderPass*, *ufoProcessCalcDefinedRectangle*, *ufoProcessCalcNeedRectangles*, *ufoProcessCalcDefinedPixelTypes*, and *ufoProcessSpecifyConvertPixelTypes*.

ufoProcessWhichField

```
int ufoProcessWhichField(
    ufoProcess        process_instance);
```

ufoProcessWhichField returns a value of 1 or 2 if field rendering is on, otherwise, it returns a value of 0.

ufoProcessFieldRendering may be called within *ufoProcessRenderPixel*, *ufoProcessRenderLine*, *ufoProcessRenderRectangle*, *ufoProcessCanAvoidRendering*, *ufoProcessPreRender*, *ufoProcessPostRender*, *ufoProcessPreRenderPass*, *ufoProcessPostRenderPass*, *ufoProcessCalcDefinedRectangle*, *ufoProcessCalcNeedRectangles*, *ufoProcessCalcDefinedPixelTypes*, and *ufoProcessSpecifyConvertPixelType*.

ufoProcessGetDefinedRectangle

```
ufoRectangle ufoProcessGetDefinedRectangle(
    ufoProcess        process_instance,
    int               input_index);
```

ufoProcessGetDefinedRectangle gets the defined maximum rectangle for the raster that can be supplied to a specific process input, regardless of the actual size of rectangle supplied.

This will be the same rectangle which is supplied to the *input_rectangles* and *needed_input_rectangle* arguments of *ufoProcessCalcDefinedRectangle* and *ufoProcessCalcNeedRectangles*.

ufoProcessGetDefinedRectangle may be called within *ufoProcessRenderPixel, ufoProcessRenderLine, ufoProcessRenderRectangle, ufoProcessCanAvoidRendering, ufoProcessPreRender, ufoProcessPostRender, ufoProcessPreRenderPass, ufoProcessPostRenderPass, ufoProcessCalcDefinedRectangle*, and *ufoProcessCalcNeedRectangles*.

ufoProcessGetDefinedRectangle may be called within *ufoProcessRenderPixel*, *ufoProcessRenderLine*, *ufoProcessRenderRectangle*, *ufoProcessPreRender*, *ufoProcessPostRender*, *ufoProcessPreRenderPass*, and *ufoProcessPostRenderPass*.

ufoProcessGetFullRenderRectangle
ufoProcessGetFullRenderRectangleStruct

```
void ufoProcessGetFullRenderRectangle(
    ufoProcess        process_instance,
    int               *x1,
    int               *y1,
    int               *x2,
    int               *y2);
void ufoProcessGetFullRenderRectangleStruct(
    ufoProcess          process_instance,
    ufoRectangle        *rectangle);
```

Refer to the following table for command descriptions:

Command	Description
`rectangle`	rectangle structure into which values copied
`ufoRectangle`	structure defined in *ufoTypes.h*.

Returns the full rectangle resolution of the final target clip in the process hierarchy being rendered, regardless of any cropping or scaling. This should be called when calculating some position or region relative to the full render region, such as a centre point.

ufoProcessGetFullRenderRectangle and *ufoProcessGetFullRenderRectangleStruct* may be called within *ufoProcessRenderPixel*, *ufoProcessRenderLine*, *ufoProcessRenderRectangle*, *ufoProcessCanAvoidRendering*, *ufoProcessPreRender*, *ufoProcessPostRender*, *ufoProcessPreRenderPass*, *ufoProcessPostRenderPass*, *ufoProcessCalcDefinedRectangle*, *ufoProcessCalcNeedRectangles*, *ufoProcessCalcDefinedPixelTypes*, *ufoProcessSpecifyConvertPixelTypes*, and from within any editor user function or event function in the editor GUI.

ufoRasterAlloc

```
ufoRaster ufoRasterAlloc(
    ufoPixelType        pixel_type,
    int                 x1,
    int                 y1,
    int                 x2,
    int                 y2);
```

Refer to the following table for command descriptions:

Command	Description
`pixel_type`	type of pixel required in new raster
`x1, y1, x2, y2`	bounds of new raster
`ufoPixelType`	
• 8-bits per component. Component range 0..255	
`ufoRGB8PixelType`	RGB packed UBGR (U is redundant) 32-bits per pixel
`ufoA8PixelType`	Alpha 8-bits per pixel
`ufoRGBA8PixelType`	RGB-Alpha packed ABGR 32-bits per pixel
• 16-bits per component. Component range 0..16383 (14-bits)	
`ufoRGB16PixelType`	RGB packed BGR 48-bits per pixel
`ufoA16PixelType`	Alpha 16-bits per pixel
`ufoRGBA16PixelType`	RGB-Alpha packed ABGR 64-bits per pixel
• Floating point 32-bits per component. Component range 0.0..1.0	
`ufoRGBFPixelType`	RGB packed BGR 96-bits per pixel
`ufoAFPixelType`	Alpha 32-bits per pixel
`ufoRGBAFPixelType`	RGB-Alpha packed ABGR 128-bits per pixel

ufoRasterAlloc allocates raster of the given size and type, and returns a pointer to the new instance.

ufoRasterAllocCopy

```
ufoRaster ufoRasterAllocCopy(
    ufoRaster         raster_instance,
    int               do_copy_data);
```

ufoRasterAllocCopy allocates a raster of the same size and type as *raster_instance*, and returns a pointer to the new instance. The pixel values can be optionally copied to the new raster.

ufoRasterAllocCopyPart

```
ufoRaster ufoRasterAllocCopyPart(
    ufoRaster         raster_instance,
    int               x1,
```

```
    int                 y1,
    int                 x2,
    int                 y2,
    int                 do_copy_data);
```

ufoRasterAllocCopyPart allocates a raster of the same type as *raster_instance*, and of the size defined in the arguments, and return a pointer to the new instance. The pixel values can be optionally copied to the new raster.

ufoRasterCopy

```
void ufoRasterCopy(
    ufoRaster           raster_instance_from,
    ufoRaster           raster_instance_to);
```

ufoRasterCopy copies one raster to another, handling any pixel conversion and bound checking required.

ufoRasterCopyPart

```
void ufoRasterCopyPart(
    ufoRaster             raster_instance_from,
    ufoRaster             raster_instance_to,
    int                   x1,
    int                   y1,
    int                   x2,
    int                   y2);
```

ufoRasterCopyPart copies a region of one raster to another, handling any pixel conversion and bound checking required.

ufoRasterFree

```
void ufoRasterFree(
    ufoRaster             raster_instance);
```

ufoRasterFree frees the raster previously allocated using either *ufoRasterAlloc*, *ufoRasterAllocCopy*, or *ufoRasterAllocCopyPart*.

ufoRasterGetBufferAddress

```
void* ufoRasterGetBufferAddress(
    ufoRaster             raster_instance);
```

ufoRasterGetBufferAddress returns a pointer to the base of the raster buffer.

ufoRasterGetBufferSize
ufoRasterGetPixelSize
ufoRasterGetComponentSize

```
int ufoRasterGetBufferSize(
    ufoRaster           raster_instance);
int ufoRasterGetPixelSize(
    ufoRaster           raster_instance);
int ufoRasterGetComponentSize(
    ufoRaster           raster_instance);
```

ufoRasterGetBufferSize returns the buffer size of the raster in bytes.

ufoRasterGetPixelSize returns the pixel size of the raster in bytes.

ufoRasterGetComponentSize returns the component size of the raster in bytes.

ufoRasterGetLimits
ufoRasterGetWidth
ufoRasterGetHeight

```
void ufoRasterGetLimits(
    ufoRaster         raster_instance,
    int               *x1,
    int               *y1,
    int               *x2,
    int               *y2);
int ufoRasterGetWidth(
    ufoRaster         raster_instance);
int ufoRasterGetHeight(
    ufoRaster         raster_instance);
```

ufoRasterGetLimits returns the bounds of a raster.

ufoRasterGetWidth returns the width of a raster. This is always x2-x1+1.

ufoRasterGetHeight returns the height of a raster. This is always y2-y1+1.

ufoRasterGetPixelAddress

```
void* ufoRasterGetPixelAddress(
    ufoRaster         raster_instance,
    int               x,
    int               y);
```

ufoRasterGetPixelAddress returns the address of the raster buffer at pixel (X,Y).

The returned pointer can be converted into the correct pixel structure for the type raster. For pixel structure and component definitions, see *ufoRaster.h*.

Note Bounds checking is not performed on X or Y.

ufoRasterGetPixelCol
ufoRasterGetPixelAlpha
ufoRasterGetPixelColAlpha

```
void ufoRasterGetPixelCol(
    ufoRaster         raster_instance,
    int               x,
    int               y,
    double*           r,
    double*           g,
    double*           b);
void ufoRasterGetPixelAlpha(
    ufoRaster         raster_instance,
    int               x,
    int               y,
    double*           a);
void ufoRasterGetPixelColAlpha(
    ufoRaster         raster_instance,
    int               x,
    int               y,
    double*           r,
    double*           g,
    double*           b,
    double*           a);
```

ufoRasterGetPixelCol returns the RGB values for a pixel.

ufoRasterGetPixelAlpha returns the Alpha value for a pixel.

ufoRasterGetPixelColAlpha returns the RGB-Alpha values for a pixel.

Values are normalized in the range (0..1).

Depending on the raster, the values returned are as follows:

ufoRasterGetPixelCol

- Called on an RGB raster returns r=R g=G b=B
- Called on an A raster returns r=A g=A b=A

- Called on an RGBA raster returns r=R g=G b=B

ufoRasterGetPixelAlpha

- Called on an RGB raster returns a = (R+G+B)/3
- Called on an A raster returns a = A
- Called on an RGBA raster returns a = A

ufoRasterGetPixelColAlpha

- Called on an RGB raster returns r=R g=G b=B a=1
- Called on an A raster returns r=A g=A b=A a=A
- Called on an RGBA raster returns r=R g=G b=B a=A

ufoRasterGetPixelType

```
ufoPixelType ufoRasterGetPixelType(
    ufoRaster         raster_instance);
```

Refer to the following table for command descriptions:

Command	Description
`ufoPixelType`	
•8-bits per component. Component range 0..255	
`ufoRGB8PixelType`	RGB packed UBGR (U is redundant 32-bits per pixel
`ufoA8PixelType`	Alpha 8-bits per pixel
`ufoRGBA8PixelType`	RGB-Alpha packed ABGR 32-bits per pixel
•16-bits per component. Component range 0.16383 (14-bits)	
`ufoRGB16PixelType`	RGB packed BGR 48-bits per pixel
`ufoA16PixelType`	Alpha 16-bits per pixel
`ufoRGBA16PixelType`	RGB-Alpha packed ABGR 64-bits per pixel
•Floating point 32-bits per component. Component range 0.0..1.0	
`ufoRGBFPixelType`	RGB packed BGR 96-bits per pixel
`ufoAFPixelType`	Alpha 32-bits per pixel
`ufoRGBAFPixelType`	RGB-Alpha packed ABGR 128-bits per pixel

ufoRasterGetPixelType returns the pixel type for the raster.

ufoRasterSetPixelCol
ufoRasterSetPixelAlpha
ufoRasterSetPixelColAlpha

ufoRasterSetPixelCol sets the RGB values in a pixel.

ufoRasterSetPixelAlpha sets the Alpha values in a pixel.

ufoRasterSetPixelColApha sets the RGB-Alpha values in a pixel.

Values should be normalized in the range (0..1).

Depending on the raster type, the values set are as follows:

ufoRasterSetPixelCol

- Called on an RGB raster sets R=r G=g B=b
- Called on an A raster sets A = (r+g+b)/3
- Called on an RGBA raster sets R=r G=g B=b A=1

ufoRasterSetPixelAlpha

- Called on an RGB raster sets R=a G=a B=a
- Called on an A raster sets A = a
- Called on an RGBA raster sets R=a G=a B=a A=a

ufoRasterSetPixelColAlpha

- Called on an RGB raster sets R=r G=g B=b
- Called on an A raster sets A = a
- Called on an RGBA raster sets R=r G=g B=b A=a

ufoEditorCreateStandardEditor

```
void * ufoEditorCreateStandardEditor(
    ufoEditor         editor_instance,
    void *            parent_window,
    int*              param_index);
```

ufoEditorCreateStandardEditor creates a standard invocation of a parameter editor within a UFO custom GUI. For example, a slider for a ufoFloatParam, an option menu for a ufoEnumParam, or a colour swatch for a ufoColourRGBParam.

For ganged parameters, you only need to call this function for the first parameter in a ganged sequence of parameters.

ufoEditorCreateStandardEditor returns the window pointer to the top-most window in created editor

The user is responsible for positioning the child window within the parent

*ufoEditorCreateStandardEdito*r may be called within *ufoEditorCreate*.

ufoEditorSetUserData

```
void ufoEditorSetUserData(
     ufoEditor         editor_instance,
     void              *user_data);
```

Sets user data to be associated with this process editor. User data may be set to null.

If editor user data should be tidied when the editor is destroyed, this should be handled in the *ufoEditorDelete* user function. *ufoEditorSetUserData* may be called within any editor user function.

ufoEditorGetUserData

```
void* ufoEditorGetUserData(
     ufoEditor         editor_instance);
```

Returns user data previously set with *ufoEditorSetUserData* for this process editor instance. *ufoEditorGetUserData* returns null if no user data is currently set.

ufoEditorGetUserData may be called within any editor user function.

ufoEditorGetOverlayIndex
ufoEditorGetOverlayIndexRange

```
int ufoEditorGetOverlayIndex(
     ufoEditor         editor_instance
     int               colour_index);
void ufoEditorGetOverlayIndexRange(
     ufoEditor         editor_instance,
     int *             min_colour_index);
     int *             max_colour_index);
```

ufoEditorGetOverlayIndex gets the correct colour index to be used for setting the overlay colours in OpenGL prior to drawing in the overlay plane. This is necessary as Media Illusion splits the overlay plane and reserves half for specific tools, requiring an offset to be taken into consideration.

ufoEditorGetOverlayIndexRange gets the range of valid index to colour values which can be set and used when drawing into the overlay.

The transparent colour index is always 0, and colour indices 1, 2, and 3 relate to user-definable colours, accessed through the application preferences interface.

ufoEditorGetOverlayIndex and *ufoEditorGetOverlayIndexRange* may be called within any editor user function.

ufoEditorSetOverlayColour

```
int ufoEditorSetOverlayColour(
    ufoEditor         editor_instance
    int               colour_index);
    double            red);
    double            green);
    double            blue);
```

ufoEditorSetOverlayColour assigns a colour to a specific overlay index.

The transparent colour index is always 0; this cannot be changed. Colour indices 1, 2, and 3 relate to user-definable colours, accessed through the application preferences interface, which should not be reassigned.

ufoEditorSetOverlayColour may be called within any editor user function.

ufoEditorGetPixelSize

```
void ufoEditorGetPixelSize(
    ufoEditor         editor_instance,
    double*           x_size,
    double*           y_size);
```

Refer to the following table for command descriptions:

Command	Description
`x_size,y_size`	x and y scale factor

ufoEditorGetPixelSize gets the scaling in the viewer from the size if a pixel at full render resolution to the size in the current display.

ufoEditorGetPixelSize may be called with any editor user function.

ufoEditorDraw

```
void ufoEditorDraw(
    ufoEditor         editor_instance);
```

ufoEditorDraw causes the editor to set up the context for OpenGL drawing and call *ufoEditorOverlayDraw* or *ufoEditorRGB Draw* if supplied.

If you need to call *ufoEditorDraw* from within event functions in the user-defined GUI, then the UFO will need to store the *ufoEditor* pointer passed to it in *ufoEditorCreateGui* when this editor instance was created.

ufoEditorDraw may be called within *ufoEditorMouseDown, ufoEditorMouseUp, ufoEditorMouseDrag, ufoEditorMousePosition, ufoEditorKeyEvent*, or from within any event function in the editor GUI.

ufoEditorNotifyEdited

```
void ufoEditorNotifyEdited(
     ufoEditor          editor_instance,
     int*               param_index_array,
     int                number_of_parameters,
ufoEditorEventType      ufo_event_type);
```

ufoEditorNotifyEdited notifies Media Illusion of changes to parameters as a result of an edit.

For interactive changes, for example when dragging sliders or onscreen tools, *ufoEditorDrag* should be passed as the *ufo_event_type*.

For release changes, such as releasing sliders or onscreen tools, *ufoEditorRelease* should be passed as the *ufo_event_type*.

ufoEditorNotifyEdited may be called within *ufoEditorOverlayDraw, ufoEditorRGBDraw, ufoEditorMouseDown, ufoEditorMouseUp, ufoEditorMouseDrag, ufoEditorMousePosition*, and *ufoEditorKeyEvent*.

ufoEditorGetRasterViewed

```
ufoRasterufoEditorGetRasterViewed(
     ufoEditor          editor_instance);
```

ufoEditorGetRasterViewed returns handle to raster instance for the currently displayed raster during editing. The raster handle can be passed in calls to raster functions which take a *ufoRaster* handle as the first argument.

All returned raster handles should be tested against null, as under certain circumstances such as prior to processing or getting from unconnected UFO process inputs, the raster may not be available. If a returned handle is null then there is no raster available to get.

ufoEditorGetRasterViewed may be called within *ufoEditorMouseDown(), ufoEditorMouseUp(), ufoEditorMouseDrag(), ufoEditorMousePosition(), ufoEditorKeyEvent(),* or from within any event function in the editor GUI.

ufoEditorSetCurrentFrame

```
void ufoEditorSetCurrentFrame(
     ufoEditor                  editor_instance,
     double                     frame,
     ufoEditorEventType         event_type);
```

ufoEditorSetCurrentFrame tells the application to change the current frame. If the event_type in is set to *ufoEditorForce*, it will ensure that the frame is rendered.

ufoEditorSetCurrentFrame may be called within *ufoEditorMouseDown( )*, *ufoEditorMouseUp( )*, *ufoEditorMouseDrag( )*, *ufoEditorMousePosition( )*, *ufoEditorKeyEvent( )*, or from within any event function in the editor GUI.

ufoEditorForceRender

```
void ufoEditorForceRender(
    ufoEditor         editor_instance);
```

ufoEditorForceRender forces the UFO process to render the current frame if the update type in the editor is set to interactive drag or pen-up.

ufoEditorForceRender may be called within *ufoEditorMouseDown( )*, *ufoEditorMouseUp( )*, *ufoEditorMouseDrag( )*, *ufoEditorMousePosition( )*, *ufoEditorKeyEvent( )*, or from within any event function in the editor GUI.

ufoEditorGetCurrentViewType

```
ufoEditorGetCurrentViewType(
    ufoEditor         editor_instance,
    ufoViewType       *io_type,
    int               *io_num);
```

Refer to the following table for command descriptions:

Command	Description
`iotype`	ufoInput, ufoOutput or ufoMask
`io_num`	index of iotype

Inquires what view type is currently set in the editor (Input, Output or Mask) and the view index if there is more than one (for example input 0 or input 1).

Returns a value of 1 if current view type is valid or 0 if it is invalid.

ufoViewType is defined in *ufoTypes.h* and is one of: *ufoInput*, *ufoOutput* or *ufoMask*.

May be called within any editor user function.

ufoEditorSetStatusBar

```
void ufoEditorSetStatusBar(
    const char *      message_string);
```

Puts a message in the application status bar. To clear the message, pass in a null string

May be called within *ufoEditorMouseDown( )*, *ufoEditorMouseUp ( )*, *ufoEditorMouseDrag ( )*, *ufoEditorMousePosition ( )*, *ufoEditorKeyEvent ( )*, or from within any event function in the editor GUI.

ufoMalloc
ufoRealloc
ufoFree

```
void* ufoMalloc(
    size_t          size);
void* ufoRealloc(
    void            *ptr,
    size_t          size);
void ufoFree(
    void            *ptr);
```

Host memory allocation routines. These have the same functionality as the standard *malloc, realloc,* and *free* system calls.

You should use the UFO memory allocation mechanism for dynamically allocating and freeing memory as it makes use of the Media Illusion memory manager.

ufoCheckInBackground

```
int ufoCheckInBackground(void);
```

ufoCheckInBackground checks if the main application is running in foreground mode (with interactive user interface) or in background mode (with no user interface).

May be called within any UFO user function

ufoMPFork

```
void ufoMPFork(
    ufoMPForkFunc     func,
    int               force_exactly_n,
    ...);
```

ufoMPFork creates threads that execute the supplied function *func* in parallel with the calling thread, here thread management is handled by the application multi-threader.

force_exactly_n should be set to 0: for standard behaviour of creating n-1 additional processes to the calling process where n is the application default number of threads used in multi-threading, and n (where n is greater than 0): to force creation of exactly n-1 additional processes.

num_args should be set to the number of arguments being passed to *func* in the following argument list. A maximum of 6 arguments can be passed.

ufoMPForkFunc typed for function to be called in parallel is supplied in *ufoTypes.h*.

ufoMPFork should not be called within a function which itself is being multi-threaded.

If *ufoProcessSetMPSafe* is set to false, then the application will not attempt to call any of the UFO functions in parallel and *ufoMPFork*.

May be called in any UFO user function. However, if *ufoProcessSetMPSafe* is set to true (the default behaviour), then *ufoMPFork* will have no effect in *ufoProcessRenderLine* or *ufoProcessesPixel*.

May be called within any UFO user function.

ufoMPForkMyId

```
int ufoMPForkMyId(void);
```

When forked processes are created with *ufoMPFork*, each is given a unique thread ID, which can be obtained via *ufoMPForkMyId*. The return value will bean in the range 0..n-1, where n is the number of threads running in parallel.

Returns -1 if the current process thread is not handled by the application multi-threader.

May be called within the *(ufoMPForkFunc)fun*c supplied to *ufoMPFork*.

ufoMPForkNumProcs

```
int ufoMPForkNumProcs(void);
```

Returns the number of threads that have been forked by *ufoMPFork*.

May be called within the *(ufoMPForkFunc)func* supplied to *ufoMPFork*.

ufoMPForkPotentialNumProcs

```
int ufoMPForkPotentialNumProcs(void);
```

Returns the number of potential threads available to processes.

This may be restricted by machine processors present, application multi-processor licensing or application command line arguments specifying number of threads to use.

This can be used to determine potential maximum number of threads available for licensing purposes during a UFO process definition.

May be called within the *(ufoMPForkFunc)func* supplied to *ufoMPFork*.